University of Calabria
Department of Mathematics and Computer Science

Master's Degree Course in Computer Science

Master's Thesis

Lazy Declarative Heuristics in Answer Set Programming: Design and Evaluation of a clingo Propagator

Supervisor at the University of Calabria:
Prof. Carmine Dodaro

Supervisors at the University of Potsdam:
Prof. Torsten Schaub
Prof. Javier Romero

Candidate:
Pierpaolo Spadafora

Student ID 263722

Academic Year 2025/2026

Contents

Abstract

Answer Set Programming (ASP) provides a declarative way to describe combinatorial problems while delegating the construction and exploration of the solution space to a grounding and solving system. In ground-and-solve systems such as clingo, the relevant part of a program is instantiated before search starts. This architecture is robust and mature, but it becomes expensive when heuristic directives generate a large number of ground instances, which happens precisely when the intended heuristic depends on the partial state of the computation, for example through an aggregate over the atoms that are currently true.

This thesis presents the design, implementation, and evaluation of a lazy heuristic propagator integrated into a modified version of clingo. The propagator keeps domain-specific heuristics in a compact, declarative form and evaluates them at solving time against the partial assignment, thereby avoiding the combinatorial materialization of large families of `#heuristic` directives. Two interchangeable evaluation backends are provided on top of the same propagation machinery: a *native* backend, in which heuristics are ground facts of the form `__heuristic(...)` interpreted by C++ runtime structures with incremental dynamic aggregates, and a *Prolog* backend, in which heuristics are rule strings of the form `heuristic("...")` evaluated by an in-process SWI-Prolog engine synchronized with the solver trail.

The evaluation is organized along two orthogonal axes. The first axis is the *approach*: ground-and-solve, where the heuristic is expanded by the grounder, versus lazy, where it is evaluated at runtime. The second axis is the *semantics of partial information*: a clingo-like reading, in which negation and aggregates refer to established falsity and to completely determined aggregate sources, versus an Alpha-like reading, in which `not a` is satisfied as soon as a is not assigned true and aggregates are computed over the atoms currently true in the partial assignment. Crossing the two axes yields the four variants `ga`, `gc`, `la`, and `lc` that are compared, together with a heuristic-free baseline, on three benchmark problems: Balanced Set Partitioning (BSP), the Partner Units Problem (PUP), and the House Reconfiguration Problem (HRP). A further technical contribution is the systematic *flattening* of Alpha's weight–priority pairs into single clingo weights, which is required because priorities rank heuristics globally in Alpha but only arbitrate modifications of the same atom in clingo.

The central experimental claim concerns memory. On BSP, the number of ground heuristic objects produced by the ground-and-solve variants grows as $\Theta(n^3)$, reaching 1,010,200 ground `#heuristic` directives at $n = 100$, whereas the lazy variants keep exactly two heuristic facts, independent of n. On PUP, the saving converts directly into reach: the unguided baseline and the plain ground heuristic stall by *timeout* at $N = 80$ under a fixed time budget, while both Alpha-like variants solve every tested instance up to $N = 200$; over that common range, the lazy variant consistently costs about half the time and half the peak memory of its ground counterpart at equal size, a gap that widens with N because the ground variant additionally pays for the unrolled heuristic directives it must materialize. On HRP, whose heuristics contain no aggregates, the comparison isolates the semantics of negation: the Alpha-like reading reproduces the intended guidance decision for decision, while the clingo-like

reading proves either inert or actively misleading. Laziness does not eliminate the cost of the dynamic heuristic: it moves it from grounding-time space to solving-time work, a trade-off that the campaign quantifies through the per-decision metrics of the propagator. The suite also includes an equivalence harness proving that all variants and both backends preserve the same answer sets, and guards that detect silent misconfiguration of the Prolog backend.

1 Introduction

1.1 Context

Answer Set Programming (ASP) is a declarative problem-solving paradigm for combinatorial problem solving. Instead of prescribing an algorithm step by step, the modeller describes the properties that admissible solutions must satisfy. The ASP solver then computes the answer sets, that is, the stable models of the program, which correspond to the solutions of the encoded problem [1, 2].

Most modern ASP systems, such clingo, follow a *ground-and-solve* architecture. A grounder first rewrites the non-ground program, which may contain variables, into an equivalent propositional program. A solver based on CDCL then explores assignments and computes stable models [2, 3]. In clingo these two phases are handle by gringo, the grounder, and clasp, the solver. The separation is effective in many domains, but it also exposes a well-known limitation: grounding may become the bottleneck whenever the propositional instance grows much faster than the original declarative specification.

Domain-specific heuristics are one of the main tools for guiding search. In clingo they can be written as directives of the form:

```
#heuristic A : Body. [W@P, M]
```

Here A is the target atom, $Body$ is the applicability condition, W is the heuristic weight, P is the priority level among directives on the same target atom, and M is the modifier. The literature shows that domain-specific heuristics can be decisive on difficult industrial instances [4, 5]. At the same time, if a heuristic directive contains aggregates or a rich body, grounding the directive may result in a combinatorial explosion that would fill the system memory. Hence, a measure used to speed up the solving process ends up to making it unreachable.

The goal of this thesis is to preserve a declarative form for heuristics, while moving part of their evaluation from grounding time to search time. The result is not a general lazy grounder for ASP. It is a specialized propagator inside clingo and instantiated in two interchangeable evaluation backends.

1.2 Grounding Bottleneck and Dynamic Heuristics

The grounding bottleneck appears when the propositional representation required by the solver becomes too large in time or memory. The phenomenon is especially visible in programs that combine numeric domains, aggregates, and conditions whose relevance depends on the evolving search state. Lazy-grounding research addresses this issue by interleaving grounding and solving, instantiating only the parts of the program that are needed during search [6].

Dynamic heuristics add a second layer of complexity. A useful heuristic choice may depend not only on the input instance, but also on the partial assignment currently built by the solver. A decision may become preferable because some atoms are already true, because other atoms are still undecided, or because an aggregate computed over true atoms has reached a favourable value. This idea is studied in Alpha [7], a lazy-grounding system in which declarative heuristics can reason about partial states and dynamic aggregates [6, 8].

The reference papers make the source of the problem concrete. Consider the heuristic used throughout this thesis for Balanced Set Partitioning:

```
#heuristic b(X) : x(X), not c(X), S = #sum{Y : c(Y)}, W = ((n+1)*S)+X. [W, true]
```

The aggregate value S appears *inside the weight*. Since the atoms `c(Y)` are choice atoms rather than facts, the grounder cannot know the value of the sum at grounding time: it must enumerate every reachable value of S and materialize a distinct ground directive for every pair (X, S). In a lazy-grounding system such as Alpha the same heuristic stays symbolic and the aggregate is simply re-evaluated over the atoms currently true. Reproducing this behaviour inside clingo, whose grounder is eager by design, is the engineering challenge addressed here.

clingo already offers a mature language for `#heuristic` directives and a Domain heuristic in the solver, but its ground-and-solve pipeline materializes the ground instances of such directives before search. When the heuristic component grows significantly, it becomes attractive to keep the heuristic in a compact representation and evaluate it at runtime.

1.3 Objective and Experimental Design

The objective of this thesis is to design and evaluate an extension of clingo that evaluates domain-specific heuristics lazily, and to compare it systematically against the ground-and-solve alternative. The comparison is organized along two orthogonal axes.

The first axis is the *approach*. In the *ground* approach (prefix `g`), the heuristic is expressed with native `#heuristic` directives and fully expanded by gringo. In the *lazy* approach (prefix `l`), the heuristic is kept as a small number of ground facts, read by a custom propagator during initialization and evaluated during search.

The second axis is the *semantics of partial information*, that is, the question of how a heuristic condition reads the current partial assignment. Under the *clingo-like* semantics (suffix `c`), negation and aggregates are evaluated as clingo would: `not a` is satisfied only when a is assigned false, and an aggregate refers to a completely determined set of source atoms. Under the *Alpha-like* semantics (suffix `a`), `not a` is satisfied as soon as a is not assigned true, and the aggregate is computed over the atoms currently true in the partial assignment, so its value evolves during search. Crossing the axes yields four variants per problem: `ga`, `gc`, `la`, and `lc`, evaluated against the heuristic-free baseline `gc_noheur`.

Two remarks are necessary to read this matrix correctly. First, everything runs on clingo: the variants labelled "Alpha" are *simulations* of Alpha's semantics inside clingo, not runs of the Alpha system. In the ground variants, the simulation is necessarily approximate, because ground clingo cannot express a sum over the atoms currently true; the `ga` encodings therefore reconstruct the dynamic aggregate through a static unrolling combined with bound conditions, as explained in Section 3.1. Only the lazy `la` variant captures the dynamic aggregate exactly. Second, Alpha and clingo disagree on the meaning of weights and priorities: in Alpha the pair (W, P) ranks heuristics globally, with priority acting as a strict level discriminator between different atoms, whereas in clingo the priority `@P` only arbitrates modifications of the same atom and the global ranking is induced by the weight alone. Faithful translation therefore requires *flattening* the levels into the weight, $W_{\mathit{flat}} = W + P \cdot M$ for a sufficiently large level multiplier M, whenever the evaluation follows clingo's reading of priorities.

The lazy approach is instantiated in two backends that share the same propagator interface. The *native* backend reads compact facts such as:

```
__heuristic(__target(b), x, __n_c, __bind(s, __sum(c)),
            __weight(__add(__mul(s, B), self)), __priority(0),
            __semantics(alpha), __modifier(true)) :- B = n + 1.
```

and evaluates them through C++ runtime templates with incremental aggregate states. The *Prolog* backend reads rule strings such as:

```
heuristic("heuristic(b(X), W, 0, true) :-
    aggregate_all(sum(Y), holds(c(Y)), S), holds(heuristic_base(Base)),
    holds(x(X)), target_available(b(X)), alpha_not(c(X)),
    W is Base*S+X.").
```

and evaluates them with an in-process SWI-Prolog engine [9] kept synchronized with the solver trail. The two backends embody two different engineering trade-offs, compiled incremental evaluation versus a general logic-programming runtime, over the same declarative content.

The thesis addresses three questions. First, it asks how a dynamic heuristic can be represented in a compact ground form that a propagator can interpret, and how the Alpha-like and clingo-like readings of partial information can coexist in a single mechanism. Second, it studies how far the clingo semantics of weights, priorities, and modifiers can be reproduced through the `Clingo::Heuristic` interface, and what the weight–priority flattening costs in terms of encoding effort. Third, it evaluates the impact of the lazy representation on grounding size, memory, and search behaviour on three benchmark problems: Balanced Set Partitioning (BSP), the Partner Units Problem (PUP) [8], and the House Reconfiguration Problem (HRP) [6].

1.4 Contributions

The implemented system consists of two modified clingo builds sharing the same source layout. The build in `clingo-native` contains the native backend: shared types, a parser for `__heuristic` facts, incremental aggregate states, and the heuristic propagator, all under `libclingo`. The build in `clingo-prolog` contains the Prolog backend: the same propagator skeleton, a rule-string normalizer, an in-process SWI-Prolog evaluation backend, and an auxiliary clingo-based fallback evaluator. In both builds the propagator is registered automatically in `clingo_app.cc`, so the modified binaries use the mechanism without any external controller.

On the experimental side, the project provides encodings of BSP, PUP, and HRP in all four variants for both backends, under `test_folder/encodings-native` and `test_folder/encodings-prolog`; an equivalence harness, `tools/check_equivalence.py`, which verifies that heuristics preserve the answer sets and that the Prolog backend is actually active; and a benchmark pipeline built on `benchmark-tool` and `runlim`, with wrapper systems for the two binaries, a result parser that extracts solver and propagator statistics, and automatic graph generation.

1.5 Structure of the Thesis

Section 2 provides the necessary background: the stable model semantics, the ground-and-solve architecture and its conflict-driven search, the `#heuristic` language of clingo, lazy grounding and the heuristic semantics of Alpha, and the propagator interface on which the implementation rests. Section 3 describes the methodology: the conceptual two-axis framework, the weight–priority flattening, the architecture of the two backends, the benchmark encodings with their adaptations, the correctness harness, and the measurement infrastructure. Section 4 presents the experimental results of the full campaign, executed on an HPC cluster under uniform resource limits: the complete analysis on BSP, the scaling study on PUP, the semantics study on HRP, and the cross-backend comparison with per-decision instrumentation. Section 5 discusses the findings, states the limitations explicitly, and outlines future work.

2 Background

This section collects the notions the rest of the thesis builds on: the language and semantics of Answer Set Programming, the ground-and-solve architecture of clingo and the conflict-driven search it performs, the `#heuristic` language for domain-specific guidance, the lazy-grounding alternative embodied by Alpha together with its heuristic semantics, and the propagator interface through which clingo can be extended without modifying the solver core. Readers familiar with ASP and clingo may skim this section and return to it for the conventions fixed here, in particular the clingo reading of weights and priorities (Section 2.3) and the Alpha reading of partial assignments (Section 2.4), which the rest of the thesis contrasts systematically.

2.1 Answer Set Programming

Answer Set Programming is a declarative programming paradigm rooted in the stable model semantics of logic programming [10, 1]. A *normal rule* has the form

$$a \leftarrow b_1, \ldots, b_m,\ not\ c_1, \ldots, not\ c_n,$$

where a, b_i, and c_j are atoms: the head a is derived whenever every positive body atom b_i is derivable and no negative body atom c_j is. The negation *not* is *default negation*: *not* c holds in the absence of a derivation for c, not because c is proven false. This makes the semantics non-monotonic, and it is defined through the notion of a stable model. Given a set of atoms X and a ground program P, the *reduct* P^X is obtained by deleting every rule whose negative body intersects X and removing the negative literals from the remaining rules; X is a *stable model* (or *answer set*) of P if X is the least model of the positive program P^X [10]. Intuitively, a stable model is a set of atoms that is both closed under the rules and justified by them: every atom in the model has a non-circular derivation that survives the model's own assumptions about what is false.

Problem solving in ASP follows the *guess-and-check* idiom. A choice construct opens the solution space, integrity constraints (rules with an empty head) prune the candidates that violate the specification, and the answer sets of the program correspond one-to-one to the solutions of the problem. The input language of modern systems extends normal rules with first-order variables, arithmetic, choice rules such as `{ b(X) } :- x(X)`, conditional literals, and aggregate atoms such as `#sum`, `#count`, `#min`, and `#max`, which condense a set of weighted contributions into a single comparable value [2]. Aggregates are central to this thesis: the heuristics under study rank decisions by expressions such as "the current sum of the elements already placed in a set", and how and *when* such an aggregate is evaluated is exactly what distinguishes the systems compared here.

2.2 Ground-and-Solve: gringo, clasp, and Conflict-Driven Search

clingo [11] couples the grounder gringo with the solver clasp into a single system. The grounder instantiates the first-order program over its Herbrand universe, producing a variable-free (propositional) program; *intelligent instantiation* restricts this expansion to rule instances whose positive body is potentially derivable, so the ground program is in general much smaller than the full instantiation, but it must still be materialized in full before search starts. The solver clasp [12] then computes the stable models of the ground program with *conflict-driven nogood learning* (CDNL), the ASP counterpart of the CDCL algorithm that underlies modern SAT solvers [3]. The solver maintains a *partial assignment*, a consistent set of literals over the program atoms; it alternates deterministic *unit propagation* with nondeterministic *decisions*, and when propagation derives a contradiction it analyzes the conflict, learns a nogood that prevents its repetition, and *backjumps* to an earlier decision level. Which atom to decide on, and with which polarity, is delegated to a *decision heuristic*; clasp's default is an activity-based scheme in the VSIDS family [3], which favours atoms that appear in recent conflicts.

Two features of this architecture matter for the thesis. First, every atom and rule the solver will ever see must exist before the first decision is made: whatever knowledge the modeller wants the heuristic to consult at runtime must be compiled, at grounding time, into propositional material. Second, the size of the ground program is the dominant memory cost of the pipeline, and when a program fragment multiplies value domains, its instantiation can grow polynomially or worse in the instance parameters. This *grounding bottleneck* is a well-known limitation of ground-and-solve systems [6], and Section 4 measures a concrete manifestation of it: heuristic directives whose weight depends on an aggregate value.

2.3 Domain-Specific Heuristics in clingo

clasp exposes its decision heuristic to the modeller through the `Domain` heuristic and the `#heuristic` directive [4]:

```
#heuristic A : Body. [W@P, M]
```

For every ground instance whose *Body* holds, the directive attaches to the atom A a *modification* with weight W, priority P, and modifier M. The modifiers refine different aspects of the decision: `sign` fixes the polarity with which A is decided, `level` places A in a ranking consulted before the default heuristic, `true` and `false` combine a level with a sign, and `init` and `factor` perturb the scores of the underlying activity-based heuristic rather than overriding it. Runs must select `--heuristic=Domain` for the directives to take effect. Domain-specific heuristics of this kind have proven decisive on hard industrial configuration problems [5].

Two aspects of the clingo semantics deserve emphasis, because the thesis returns to them repeatedly. First, the priority `@P` does *not* rank different atoms against each other: it arbitrates between multiple modifications targeting the *same* atom, with the highest priority winning; the global order in which atoms are decided is induced by the resolved level value, that is, by the weights. Second, the *Body* of a ground directive is evaluated with respect to *established* truth: a condition `not a` contributes only once a is assigned false, and an aggregate has a value only when its source atoms are determined. Both conventions are natural in a ground-and-solve pipeline, and both differ from the lazy-grounding reading described next; the pair of differences generates the two-axis experimental design of Section 3.

2.4 Lazy Grounding, Alpha, and Heuristics over Partial Assignments

Lazy grounding attacks the grounding bottleneck at its root by interleaving grounding and solving: rule instances are produced during search, when their positive bodies become satisfied by the current assignment, instead of being materialized up front. Alpha [7] is the reference system of this family: it combines lazy grounding with a CDNL-style search loop, so that it retains conflict learning while never holding the full ground program in memory.

Alpha's position in this thesis, however, is not that of a competing solver but that of a *semantic* reference. Comploi-Taupe et al. extended Alpha with declaratively specified domain-specific heuristics that are evaluated against the *partial* assignment maintained during solving [6]. In their semantics, a negative heuristic condition *not a* is satisfied as soon as a is *not assigned true*, so a heuristic can fire on undecided atoms rather than waiting for established falsity; and heuristic annotations (W, P) form *global levels*: every applicable heuristic with priority P strictly precedes every heuristic with lower priority, regardless of weights, with the weight discriminating inside a level. A subsequent extension added *dynamic aggregates* to the heuristic language [8]: aggregate expressions in heuristic conditions are re-evaluated over the atoms currently true in the partial assignment, so their value evolves as search proceeds. Dynamic aggregates make heuristics such as "prefer the element whose insertion keeps the two sums balanced" directly expressible, and the two papers demonstrate the approach on configuration benchmarks, including the Partner Units Problem [13] and the House Reconfiguration Problem [14] used in this thesis.

The gap this thesis addresses is now visible: clingo offers a mature, efficient ground-and-solve pipeline whose heuristic conditions read established truth and whose aggregates are frozen at grounding time; Alpha offers heuristics that read the partial assignment and re-evaluate aggregates dynamically, but inside a different, lazily grounded system. Reproducing the second reading inside the first architecture, without paying the ground materialization that a faithful ground simulation requires, is the subject of Sections 3 and 4.

2.5 Extending clingo: the Propagator and Heuristic Interfaces

clingo can be extended at solving time through *propagators*, user-supplied components registered with the solver that participate in the search loop [11]. A propagator implements four callbacks. During `init` it inspects the symbolic atoms of the ground program, maps the atoms it cares about to solver literals, and registers *watches* on them; during search, `propagate` is invoked when watched literals become true, `undo` when backtracking retracts them, and `check` on total assignments. Watches make the mechanism incremental: the propagator is only woken up on the trail changes it declared interesting, and can maintain its own state in lockstep with the solver's partial assignment.

The `Clingo::Heuristic` interface extends this contract with a `decide` callback, invoked at each decision point *before* the solver's built-in heuristic: the callback receives the current assignment and either returns a signed literal, which becomes the next decision, or returns 0 to fall back to the default heuristic. A propagator implementing this interface can therefore observe the evolving partial assignment through its watches and steer the search through its decisions, all against the public API and without modifying clasp itself. This pair of interfaces is the mechanical foundation of the present work: the lazy heuristic mechanism of Section 3 is exactly a heuristic propagator that keeps declarative heuristic descriptions in memory, evaluates them incrementally against the trail, and answers `decide` calls with the best applicable heuristic target.

3 Methodology

3.1 Conceptual Framework: Two Axes, Four Variants

Every experiment in this thesis compares encodings of the same problem that differ only in how the domain-specific heuristic is represented and interpreted. The comparison is structured by two orthogonal axes.

The *approach* axis distinguishes where the heuristic is evaluated. In the ground-and-solve approach, the heuristic is written with native `#heuristic` directives; gringo expands them before search, and clasp's Domain heuristic consumes the resulting ground objects. In the lazy approach, the heuristic remains a handful of ground facts; a custom propagator reads them at initialization and evaluates them at every decision point against the partial assignment.

The *semantics* axis distinguishes how a heuristic condition reads partial information. Under the clingo-like semantics, conditions are evaluated with respect to *established* truth values: a negative literal `not a` holds only if a has been assigned false, and an aggregate refers to the value it has once its source atoms are completely determined. Under the Alpha-like semantics, conditions are evaluated *optimistically* on the partial state: `not a` holds as soon as a is not assigned true, and an aggregate is computed over the atoms currently true, so its value changes as search proceeds. The Alpha reading is what makes heuristics genuinely *dynamic*: a directive such as "prefer the element whose insertion keeps the sums balanced" only makes sense if the sum is read off the evolving assignment.

Crossing the axes yields the four variants used throughout, summarized in Table 1, plus the baseline `gc_noheur`, which contains the domain logic without any heuristic.

	clingo-like semantics	**Alpha-like semantics**
Ground-and-solve	`gc` (native `#heuristic`)	`ga` (simulated, see §3.1)
Lazy (propagator)	`lc`	`la`

Table 1: The experimental matrix. Each cell exists for both evaluation backends (native and Prolog) and for each benchmark problem (BSP, PUP, HRP).

Two structural constraints shape the matrix. First, ground clingo cannot express the Alpha semantics exactly: there is no ground construct denoting "the sum of the atoms currently true". The `ga` variants therefore *simulate* the Alpha reading, as described next, and only `la` captures it exactly. Second, the pair `gc`/`ga` is kept symmetric to the pair `lc`/`la`: within each approach, the two semantics differ exclusively in the treatment of negation and aggregates, so that any performance difference between suffixes can be attributed to the semantics and any difference between prefixes to the approach.

Simulating Alpha in Ground clingo

The `ga` encodings apply two systematic transformations to `gc`. The first transformation drops the negative literals on the partial state (for BSP, `not c(X)` and `not b(X)`; for PUP, the `not not_assigned_...` conditions; for HRP, the guards such as `not fullCabinet(C)`). In Alpha these literals would also be satisfied on undecided atoms, so removing them is the closest ground approximation of the Alpha reading; the solver anyway never re-decides atoms that are already assigned.

The second transformation replaces exact aggregate bindings by *bounds over a static domain*. For BSP:

```
sum_value(0..tot).
#heuristic b(X) : x(X), sum_value(S), S <= #sum{Y : c(Y)},
    W = ((n+1)*S)+X. [W, true]
```

Instead of binding S to the final value of the sum, the directive is unrolled over all candidate values `sum_value(S)` and guarded by the monotone condition $S \leq$ $\#\texttt{sum}\{Y : c(Y)\}$. During search, as soon as the partial sum reaches a bound S, every directive with that bound becomes applicable; among the applicable ground directives for the same atom, clasp's Domain heuristic retains the one with the maximum weight, which corresponds exactly to the *current* value of the growing aggregate. The max-weight selection thus reconstructs the dynamic aggregate at solving time. For aggregates whose contribution to the weight is decreasing rather than increasing (the free-capacity terms N_0, N_1, N_2 in PUP), the same trick is applied with the dual bound $V \geq \#\texttt{max}\{\ldots\}$, so that the max-weight selection picks the smallest proven value. The price of the simulation is precisely the combinatorial explosion measured in Section 4: one ground directive per point of the product of the unrolled domains.

Weights, Priorities, and Flattening

Alpha and clingo interpret the annotation $[W@P]$ differently, and the difference is central to this work. In Alpha, heuristic annotations (W, P) form global levels: every applicable heuristic with priority P strictly precedes every heuristic with priority $P' < P$, regardless of weights, and the weight discriminates within a level. In clingo, the priority `@P` only arbitrates between multiple heuristic modifications *of the same atom*; the global ranking among different atoms is induced by the resolved level value, that is, by the weight alone.

Encodings written for Alpha therefore cannot be transplanted into clingo verbatim. Whenever an ordering between different atom families is intended (zones before sensors in PUP; reuse before cabinet-filling before room-filling in HRP), the levels must be *flattened into the weight*:

$$W_{flat} = W + P \cdot M, \qquad M > \max W + 1,$$

where M is a level multiplier strictly larger than any intra-level weight, so that no heuristic of a lower level can ever outrank a heuristic of a higher level. In the PUP encodings, M is the constant offset 100,000 added to the zone weights (the maximum sensor weight is bounded by 46,220 on the benchmark instances); in HRP, M is computed from the instance as `levelMul(LM) :- LM = MC + MR + 10` over the maximum cabinet and room identifiers.

The flattening rule interacts with the two axes as follows, and this interaction is enforced consistently across the suite. In the ground variants (`gc`, `ga`), clasp's Domain heuristic is the consumer, so flattening is always required. In the lazy variants the consumer is the propagator, whose ranking behaviour is part of the semantics being modelled: under `__semantics(alpha)` the native backend ranks candidates globally by (priority, weight), faithfully reproducing Alpha's levels, so `la` encodings may use native priorities directly; under `__semantics(clingo)` the native backend deliberately reproduces clingo's local-only reading of priorities, so `lc` encodings must flatten their weights exactly like `gc`. The Prolog backend, whose selection rule is a global (priority, weight) sort, expresses the same intended order through its priority field. The net effect is that all four variants of a problem implement the same intended decision order, each through the mechanism appropriate to its semantics.

A final convention concerns empty aggregates: `#max` over an empty set is 0 in Alpha but #*inf* in clingo. All encodings therefore use `#max{0; ...}` with an explicit neutral element, and both lazy backends implement the same convention (the native `MaxState` returns 0 when empty; the Prolog runtime defines `dyn_max` with a 0 default).

3.2 System Architecture

The project maintains two modified copies of clingo with the same source layout. The build in `clingo-native` implements the native backend; the build in `clingo-prolog` implements the Prolog backend and is configured with `CLINGO_USE_SWIPL=ON` so that the SWI-Prolog engine is linked in-process. In both builds the extension lives entirely under `libclingo` and is registered in `clingo_app.cc`:

```
HeuristicPropagator my_propagator;
ctl.register_propagator(my_propagator, true);
```

The class implements `Clingo::Heuristic`, the propagator interface that additionally exposes the `decide` callback. Both backends follow the same lifecycle: during `init` they scan the symbolic atoms for their trigger facts (`__heuristic(...)` in the native build, `heuristic("...")` in the Prolog build), build their runtime representation, and register watches on the solver literals they need to observe; during search, `propagate` and `undo` keep the internal state synchronized with the trail; at each decision point, `decide` either returns a signed literal for the best applicable heuristic target or returns 0, leaving the decision to the solver's fallback heuristic. Runs are launched with `--heuristic=Domain`. Heuristics never change the answer sets: the propagator adds no constraints and only influences the order in which the solver explores the search space.

3.3 The Native Backend

Declarative Syntax

A native lazy heuristic is one ground fact per heuristic rule. The BSP heuristic, whose native directive would ground into $\Theta(n^3)$ objects, becomes:

```
__heuristic(__target(b), x, __n_c, __bind(s, __sum(c)),
            __weight(__add(__mul(s, B), self)), __priority(0),
            __semantics(alpha), __modifier(true)) :- B = n + 1.
```

The arguments are, in order: the target predicate; an implicit positive body predicate (`x`, matched on the target tuple); a negative body predicate (`__n_c` encodes `not c(X)` on the target tuple); an aggregate binding that introduces the runtime variable `s` as the incrementally maintained sum over the source predicate `c`; the weight expression, built from the operators `__add`, `__sub`, `__mul` over constants, bound variables, and the reserved symbol `self` (the first numeric argument of the target); the priority expression; the semantics selector; and the modifier. The ASP body `:- B = n + 1` is a small but important idiom: the fact is ground, and gringo evaluates the constant B and injects it as a plain number, so the propagator never needs to know program constants.

Richer bodies are expressed explicitly. When a positive body predicate does not share the target tuple, `__body(pred, __match(src, tgt), __bind_arg(var, idx))` specifies the argument correspondence and extracts values into runtime variables; `__bind_target_arg(var, idx)` names a target argument. Aggregates support per-target filtering with offsets, as in

```
__bind(m1, __max(num_sensors_on_unit, 0, __filter(1,1,-1)))
```

which, for a target `assigned_zone_unit(Z,U)`, maintains the maximum of the first argument of `num_sensors_on_unit(N,U')` restricted to source atoms with $U' = U - 1$. Since the aggregate machinery draws from a single source predicate, joins occurring inside Alpha-style aggregates are precompiled as auxiliary derived predicates in the encoding (for PUP, `sensor_assigned_zone/2` and `sensor_zone_on_unit/3`); these auxiliaries appear in no constraint, so they cannot change the answer sets.

Parser and Runtime Representation

The parser in `libclingo/src/heuristic_parser.cc` turns each `__heuristic` symbol into a `HeuristicRuleTemplate`: target predicate, positive body specifications, negative body predicates, variable bindings, modifier, semantics, and two arithmetic ASTs for weight and priority. Malformed syntax, duplicate variables, unknown operators, or negative indices are rejected eagerly with specific error messages, so that a misspelled heuristic fails fast instead of silently guiding nothing.

During `init`, the propagator builds an index from predicate names to the ground literals of their numeric tuples, and *materializes candidates*: for every ground target atom, and for every combination of matching positive body atoms, it creates a `RuntimeHeuristicCandidate` storing the target literal, the target tuple, the body literals, the values bound from body arguments, and the instantiated aggregate keys. Watches are registered on both polarities of the target and negative body literals, and on the positive body and aggregate source literals. Candidate materialization is proportional to the number of ground targets, but crucially it happens inside the propagator's memory as small POD structures, not as printed ground directives with their symbolic weights, and it does not multiply with the value domains of the aggregates.

Incremental Aggregates

Aggregate states implement a minimal interface with `add`, `remove`, and `result`. Sums and counts are plain accumulators; minima and maxima are ordered multisets, so that removal on backtracking is exact. Each source atom contributes its numeric value to the `RuntimeAggregateKey` obtained by instantiating the filter values; contributions are applied in `propagate` when a source literal becomes true and reverted in `undo`. Aggregates read under the clingo semantics are additionally *gated*: the propagator tracks the full set of source literals per runtime key and considers the aggregate usable only when every source is determined, which is the clingo-like reading of aggregate values. Under the Alpha semantics the current value is always usable.

Evaluation and Decision

A candidate is active when its target is still free, all positive body literals are true, every negative body literal is satisfied according to the template's semantics,

```
static bool negative_literal_is_satisfied(HeuristicSemantics s,
↪  Clingo::TruthValue v) {
    return s == HeuristicSemantics::Clingo ? v == Clingo::TruthValue::False
                                           : v != Clingo::TruthValue::True;
}
```

and its variable environment can be built, which for clingo-semantics templates includes the aggregate gating above. Active effects are folded per target into a `TargetHeuristicState`. The modifiers follow clingo's Domain heuristic: `level` sets the ranking value, `sign` the preferred polarity, `true`/`false` set both; `init` and `factor` are recognized but rejected, because they modify solver-internal scores that the `decide` interface cannot reach, and mapping them to `level` would be semantically misleading.

The per-target resolution and the global ranking implement the two readings of priority discussed in Section 3.1. For clingo-semantics effects, the local priority arbitrates between modifications of the same target, and the target enters the global ranking with its resolved weight only. For Alpha-semantics effects, the best effect per target is selected by (priority, weight), and the pair itself becomes the global rank of the target, reproducing Alpha's levels. Ranked targets live in an ordered set:

```
std::set<DecisionRankKey, DecisionRankGreater> active_decision_ranks_;
```

ordered by decreasing priority, then decreasing weight, then a deterministic literal tie-break. `decide` pops stale entries lazily (targets that have been assigned, or whose state changed since ranking), applies the resolved sign to the best valid target, and otherwise falls back to the solver. `undo` uses `noexcept` refresh paths throughout, since exceptions during backtracking would violate the solver's expectations.

3.4 The Prolog Backend

Rule Strings and Normalization

In the Prolog backend, a heuristic is a rule string whose head is `heuristic(Target, Weight, Priority,`

```
heuristic("heuristic(b(X), W, 0, true) :-
    aggregate_all(sum(Y), holds(c(Y)), S), holds(heuristic_base(Base)),
    holds(x(X)), target_available(b(X)), alpha_not(c(X)),
    W is Base*S+X.").
```

The body is genuine Prolog. The propagator collects all `heuristic/1` facts (the alias `prolog_heuristic/1` is accepted for compatibility), normalizes each string, and classifies its semantics syntactically: a rule containing `clingo_not(...)` is clingo-like, otherwise it is Alpha-like. To keep the synchronized state small, the propagator statically extracts the predicate signatures that actually occur in the rules (inside `holds`, `alpha_not`, `clingo_not`, `target_available`, and as rule targets) and synchronizes only atoms with those signatures.

The Runtime Program

At initialization the backend boots the SWI-Prolog engine in-process [9] and consults a generated runtime program that defines the evaluation vocabulary:

```
holds(A) :- true_atom(A).
holds(A) :- static_atom(A), \+ true_atom(A).
clingo_not(A) :- false_atom(A).
alpha_not(A)  :- \+ holds(A).
target_available(A) :- target_atom(A), \+ true_atom(A), \+ false_atom(A).
dyn_max(Goal, Template, Max) :-
    (aggregate_all(max(Template), Goal, R) -> Max = R ; Max = 0).
```

These few clauses are the entire semantic bridge. The solver trail is mirrored by the dynamic predicates `true_atom/1` and `false_atom/1`; a free atom is represented by absence from both. `alpha_not` and `clingo_not` implement exactly the two negation readings of Section 3.1; `aggregate_all` over `holds` computes dynamic aggregates on the atoms currently true, which is the Alpha aggregate semantics; `dyn_max`/`dyn_min` add the empty-set-is-zero convention; `target_available` guarantees that the rule only proposes targets that are still free.

Synchronization and Decision

Trail synchronization is incremental: `propagate` and `undo` update only the atoms mapped by the changed literals, each with a single combined retract-and-assert goal, since term parsing and calling dominate the synchronization cost. At each `decide`, the backend enumerates the solutions of `heuristic(T, W, P, M)`, discards candidates whose target is assigned or unmapped, and selects the best candidate by decreasing priority, then decreasing weight, then a deterministic tie-break; the modifier gives the sign of the returned literal. The backend is enabled by the environment variable `LAZY_HEURISTIC_BACKEND=prolog`; with `LAZY_PROLOG_STATS=1` it prints a summary line with decision counts and cumulative timings of every phase (state synchronization, Prolog query, candidate scan, selection), which the benchmark parser ingests.

Without the environment variable, the same build falls back to an auxiliary evaluator that re-grounds a small clingo program per decision. This fallback accepts a different, ASP-style body syntax; a Prolog-style rule evaluated by it simply fails (SWI's `unknown=fail` convention), so the run silently degenerates to no heuristic at all. This failure mode is invisible in exit codes and therefore dangerous for benchmarking; the guards that detect it are described in Section 3.6.

3.5 Benchmark Problems and Encodings

Balanced Set Partitioning (BSP)

BSP partitions $\{1, \ldots, n\}$ into two sets `b/1` and `c/1` whose sums differ by at most one. The guessing part is a symmetric even loop, the check compares the two sums, and the heuristic prefers to place the next element into the set whose current sum is largest, thereby keeping the partition balanced greedily. Besides the four matrix variants and the baseline, BSP has three methodological extras: `ga_weak`, which drops the negative literals without the static unrolling and therefore changes only the activation moment; `la_aux`, which routes the lazy heuristic through an auxiliary predicate `h_b(X,S)` that reintroduces grounded aggregate values (and hence a grounding cost) before the propagator is even used; and `la_co`, which combines the lazy heuristic with a linear reformulation of the balance constraint, and is treated as a modelling experiment rather than a heuristic comparison.

Partner Units Problem (PUP)

PUP [13] connects zones and sensors to communication units under adjacency and capacity constraints; the instances (the `double` family; the related `doubleV` family is left to future work) provide `comUnit/1` and `zone2sensor/2`. The encoding follows the Alpha evaluation encoding with an inductive, locality-based choice structure. The heuristic ranks sensor placements by a four-component weight (dynamic degree, forbidden placements, unit occupancy, direct connections) and zone placements by occupancy and free capacity, with zones globally preceding sensors.

Porting this encoding from Alpha to clingo required documented adaptations, applied uniformly to *all* variants so that comparisons remain fair: the inductive `assignable` rules are bounded by `comUnit(U±1)`, because gringo would otherwise materialize an unbounded chain that Alpha explores lazily; the existential capacity constraints are rewritten as monotone counting aggregates with linear grounding; a redundant distance-two blocking rule with cubic grounding is removed; counters are capped at the unit capacity (`sensorNumbers(0..2)`), without which the auxiliary predicate `num_forbidden_places_of_sensors` would explode in every variant and mask the effect under study. The zones-before-sensors ordering is flattened into the weight in `gc`, `ga`, and `lc` (offset +100,000), and expressed with a native priority in `la`, per the rule of Section 3.1.

House Reconfiguration Problem (HRP)

HRP [14] extends the House Configuration Problem with legacy configurations: things must be stored in cabinets and cabinets placed in rooms owned by the things' owners, under capacity constraints refined by the thing-length/cabinet-size extension, while a legacy configuration can be partially reused or dismantled. The heuristic is a strict five-level policy: reuse legacy components first (with internal weights), then fill cabinets with long things, then with short things, then place cabinets into rooms, and finally a closing level that biases all remaining choice atoms to false, pruning the tail of the search. HRP is the negation-heavy benchmark: its heuristics contain no aggregates, so the `la`/`lc` comparison isolates the effect of the negation semantics alone, while in BSP and PUP both mechanisms act together. In the native lazy encodings, the guards over the partial state are precompiled into same-tuple auxiliary predicates (for example `cabFull(C,T) :- cabinetDomain(C), thing(T), fullCabinet(C)`) so that negative conditions align with the target tuple; the five levels are native priorities in `la` and are flattened with the instance-derived multiplier `levelMul` in `gc`, `ga`, and `lc`.

3.6 Correctness Validation

Heuristics must not change the answer sets: this is a falsifiable property, and the pipeline verifies it before measuring anything. The harness `tools/check_equivalence.py`, run by the sanity entry point ahead of every campaign, performs an *exhaustive equivalence* check on a small BSP instance: it enumerates all models with `clingo -n 0` for every variant and both backends, projects the models onto the solution predicates (`b/1`, `c/1`), and requires the projected collections to be identical across variants and across backends. Projection is necessary because the encodings expose different `#show` sets; comparing raw witnesses would produce false positives. For PUP, where exhaustive enumeration is intractable even on small instances, the harness checks agreement on SAT/UNSAT and reports the choice counts.

The harness also guards against the silent-fallback failure mode of the Prolog backend. Every lazy Prolog run must report `decide_calls > 0` in the propagator summary: a run that terminated successfully but never called the custom `decide` has degenerated to the no-heuristic baseline and is treated as an error by the harness (and flagged with an explicit warning column by the benchmark runner). As a cross-check, the harness verifies that native `la` and Prolog `la` perform the same number of choices on the control instance: if the SWI backend were inactive, the Prolog run would diverge from the native one. The benchmark wrapper for the Prolog system exports `LAZY_HEURISTIC_BACKEND=prolog` itself, so the guard protects against configuration drift rather than repairing it. The C++ unit tests complete the picture on the native side: they cover incremental aggregates, corner cases (empty domains, unsatisfiable instances), explicit body and target bindings, filtered aggregates, the local-versus-global reading of priorities in the two semantics, and the syntax validation of the parser.

3.7 Benchmark Infrastructure and Metrics

The campaigns are orchestrated with `benchmark-tool`, driving one run per (system, setting, instance) triple under `runlim` [15] for wall-clock and memory limits. The two systems are shell wrappers around the two modified binaries; both fix `--stats=2 --heuristic=Domain -n 1` and a numeric locale, and the Prolog wrapper additionally exports the backend activation and statistics variables. A custom result parser extracts, for every run: outcome and times (total, solving, and their difference as an estimate of the non-solving part), search statistics (choices, conflicts, restarts), grounding-size measures (ground `#heuristic` directives for the ground variants, lazy heuristic facts for the lazy ones, and rule counts from clasp's statistics), the peak memory, and, for the Prolog backend, the propagator summary (decision calls, candidates seen per decision, cumulative synchronization, query, scan, and selection times).

Two measurement caveats are declared up front, because they define what the numbers can and cannot show. First, the memory metric is the peak resident set size of the whole solver process, end to end: for the lazy Prolog variants it includes the in-process SWI engine and its asserted database. The engine has a fixed initialization overhead independent of n, so on small instances the lazy variants may show a *larger* RSS than the ground ones; the lazy memory advantage is asymptotic, emerging where the $\Theta(n^3)$ growth of ground heuristics overtakes the constant overhead. Second, the ground-size comparison counts objects of different kinds: for ground variants it counts directives actually materialized by gringo (a faithful one-line-one-object count), whereas for lazy variants it counts template facts that expand at runtime into one candidate per ground target. The line count therefore *understates* the runtime work of the lazy backends by design; that work is measured separately by the per-decision metrics. Both caveats are taken up again in the discussion of the results.

4 Results

4.1 Functional Validation

The system was validated by keeping three levels separate. The first level concerns the ASP program itself: the answer sets, the `SAT`/`UNSAT` result, and the visible solutions must remain consistent across the compared encodings and across the two backends. The second level concerns the heuristic translation: targets, bodies, weights, priorities, modifiers, and aggregate bindings must be preserved when a native `#heuristic` directive is rewritten as a lazy fact, including the weight–priority flattening where the clingo reading of priorities applies. The third level concerns the effect on search, measured through choices, conflicts, solving time, memory, and the size of the ground heuristic component.

Heuristics do not change the semantics of the ASP program: they guide search, but they do not add or remove answer sets. The equivalence harness of Section 3.6 verifies this property exhaustively on BSP, by enumerating all models of every variant on both backends and comparing the collections after projection on `b/1` and `c/1`, and by SAT/UNSAT agreement on PUP, where exhaustive enumeration is intractable. The same harness enforces the anti-fallback guards for the Prolog backend (`decide_calls > 0` for every lazy Prolog run, and equal choice counts between native and Prolog `la` on the control instance). On the native side, the C++ test suite isolates the correctness of the heuristic machinery from the benchmarks: it covers incremental aggregate states, empty and unsatisfiable corner cases, explicit body and target bindings, filtered aggregates, the local-versus-global interpretation of priorities under the two semantics, and the parser's rejection of malformed syntax.

4.2 Experimental Setup and Dataset

The campaign reported in this section was executed on an HPC cluster under SLURM, with one job per (system, setting, instance) triple orchestrated by `benchmark-tool`. Every run was limited by `runlim` to 600 seconds of real time and 30 GiB of resident memory, on 4 dedicated cores. Both backends were compiled on the compute nodes against the same toolchain, and the Prolog backend embeds a modern SWI-Prolog (10.0.2) built in user space for the same environment. The dataset covers the three families with their canonical instance ranges: BSP with $n = 10$ to 200 in steps of 10; PUP with the `double-$N$` instances, $N = 20$ to 200 in steps of 20 communication units; HRP with the reconfiguration instances `house-$N$`, $N = 2$ to 20 persons in steps of 2. All five shared settings run on all families; the three methodological extras (`ga_weak`, `la_aux`, `la_co`) are BSP-specific. The grand total is 520 runs (320 for BSP, 100 each for PUP and HRP), none of which ended in error: every unsolved run is an explicit timeout (T) or memout (M). Under this raised budget, memouts become rare across the suite (they still occur at the top of the BSP range, $n \geq 180$, where the base program itself approaches the ceiling): the dominant failure mode throughout the campaign is timeout, and the frontiers reported below should be read accordingly, as time-bound rather than memory-bound except where stated.

The figures of this chapter follow the same separation. Every comparative plot shows the reference encoding and the four variants under study, `gc_noheur`, `gc`, `ga`, `la`, and `lc`; the three exploratory settings are kept out of these plots on purpose, because each of them answers a different, narrower question, and appear only in the dedicated analyses of Sections 4.8, 4.9, and 4.10, in figures restricted to the variants that each comparison actually needs.

Two properties anchor the validity of the numbers. First, within each system the ground variants (`g*`) execute identical binaries on identical ground programs, and clasp's search is deterministic at fixed configuration: choice and conflict counts are therefore exact structural measures, not noisy samples, and they reproduce bit-for-bit the counts observed in the preliminary local campaign (for example, `gc` at $n = 100$ performs exactly 32,514 choices on both machines). Second, the anti-fallback guard confirmed `decide_calls > 0` for every solved lazy Prolog run, so no lazy measurement silently degenerated to the baseline.

Tables 2–4 summarize the outcome of every cell of the experimental matrix. The columns report how many instances were solved and where the first failure occurs, distinguishing timeouts from memouts.

	native		**Prolog**	
Variant	**solved**	**first failure**	**solved**	**first failure**
`gc_noheur`	15/20	T@160	15/20	T@160
`gc`	13/20	T@140	13/20	T@140
`ga`	6/20	T@70	6/20	T@70
`ga_weak`	14/20	T@160	14/20	T@160
`la`	16/20	T@180	16/20	T@180
`lc`	15/20	T@170	15/20	T@170
`la_aux`	6/20	T@70	3/20	T@40
`la_co`	20/20	—	20/20	—

Table 2: BSP campaign overview ($n = 10..200$, 600 s / 30 GiB per run). At $n = 180$ every listed variant except `la_co` fails with both flags set (T and M simultaneously), the one point in the whole campaign where the base program's own growth exhausts the memory budget at essentially the same size at which it would time out.

	native		Prolog	
Variant	solved	first failure	solved	first failure
`gc_noheur`	4/10	T@100	4/10	T@100
`gc`	3/10	T@80	3/10	T@80
`ga`	10/10	—	10/10	—
`la`	10/10	—	10/10	—
`lc`	4/10	T@80 (non-mon.)	2/10	T@60

Table 3: PUP campaign overview (`double-`N, $N = 20..200$, 600 s / 30 GiB per run). Under this budget neither `ga` nor `la` fails anywhere in the tested range; the two ground-heuristic-free variants (`gc_noheur`, `gc`) still stall by timeout at the same sizes as before. Note that `lc` is not monotone in N: the native run fails on `double-80` but solves `double-100`.

	native		Prolog	
Variant	solved	first failure	solved	first failure
`gc_noheur`	10/10	—	10/10	—
`gc`	10/10	—	10/10	—
`ga`	10/10	—	10/10	—
`la`	10/10	—	10/10	—
`lc`	8/10	T@18	7/10	T@16

Table 4: HRP campaign overview (`house-`N, $N = 2..20$ persons, 600 s / 30 GiB per run).

Three headline facts are already visible at this granularity and are unpacked in the following subsections. The Alpha-like lazy variant `la` is the most robust setting of the whole matrix: on PUP and HRP it never times out, and on BSP it reaches the furthest of any heuristic variant, failing only at $n = 180$, exactly where the base program itself (visible already on the heuristic-free baseline two sizes earlier) starts exhausting the raised memory budget; no heuristic variant, lazy or ground, solves a BSP instance that `la` does not also solve. The ground Alpha simulation `ga`, which was designed as the faithful ground counterpart of `la`, collapses on BSP precisely because of the static unrolling that makes it faithful, and is by a wide margin the earliest variant to fail on that family. Finally, the clingo-like lazy variant `lc` exhibits two distinct failure modes, inertness on BSP and active misguidance on HRP, that the per-decision instrumentation makes directly measurable.

4.3 Reduction of the Ground Heuristic Component

The central claim of the thesis concerns *where* memory is spent, and the direct evidence is the number of ground heuristic objects as a function of n. The mechanism is readable in the encoding itself. The standard directive

```
#heuristic b(X) : x(X), not c(X), S = #sum{Y : c(Y)},
```

```
W = ((n+1)*S)+X. [W, true]
```

carries the aggregate value S *as a variable inside the weight*. Since the atoms `c(Y)` are choice atoms, the grounder cannot know the sum at grounding time and must materialize one directive for every reachable pair (X, S). With $X \in \{1, \ldots, n\}$ and S ranging over the subset sums of $\{1, \ldots, n\}$, which is every integer from 0 to $n(n+1)/2$ because 1 belongs to the domain, the count is

$$2 \cdot n \cdot \left(\frac{n(n+1)}{2} + 1\right) \;=\; \Theta(n^3),$$

where the factor 2 accounts for the two symmetric rules for `b` and `c`. The formula is verifiable exactly on the measured data: for $n = 10$ it yields $2 \cdot 10 \cdot 56 = 1{,}120$, which coincides with the measured `ground_heuristics` value for `gc`. The explicit unrolling of `ga` materializes the same (X, S) product by construction. The lazy variants remove this enumeration at the root: the aggregate is evaluated at solving time on the current partial assignment, and the materialized heuristics remain constant, two facts, independent of n.

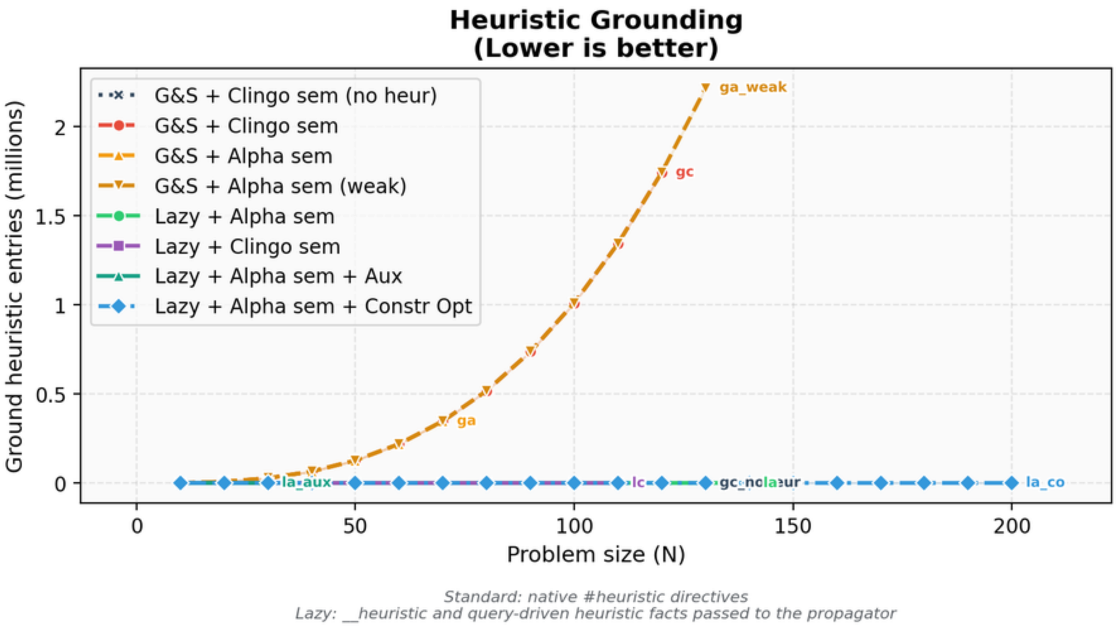

Figure 1: Comparison between native ground heuristic objects and lazy heuristic facts.

n	**Native heuristic objects**	**Lazy facts**	**Ratio**
10	1,120	2	560×
50	127,600	2	63,800×
100	1,010,200	2	505,100×
110	1,343,320	2	671,660×

Table 5: Growth of the native heuristic component compared with the lazy representation. The counts are properties of encoding and instance, independent of the machine.

This comparison must be read for what it is. For the ground variants the count is faithful: one line is one materialized object. For the lazy variants the two counted facts are *templates* that expand at runtime into one candidate per ground target (n candidates per rule for BSP); those candidates never pass through the grounder and therefore appear in no line count. The single-figure comparison "2 facts versus $\Theta(n^3)$ directives" is thus the proof of a saving in *grounding space*, not of a saving in total work: the work moved to runtime is real and is measured separately in Section 4.7.

4.4 BSP: Runtime Behaviour

Figure 2 reports the total time as measured by clingo, and Figure 3 its solving component. The base program dominates the picture for every same-encoding variant: at $n = 120$ the domain encoding grounds into 52.8 million rules, and the grounding component (about 160 seconds at $n = 120$) is essentially identical for `gc_noheur`, `la`, and `lc`, which share the encoding verbatim except for the heuristic representation. What separates the variants is the solving component and, for the ground heuristics, the additional grounding of the directives themselves.

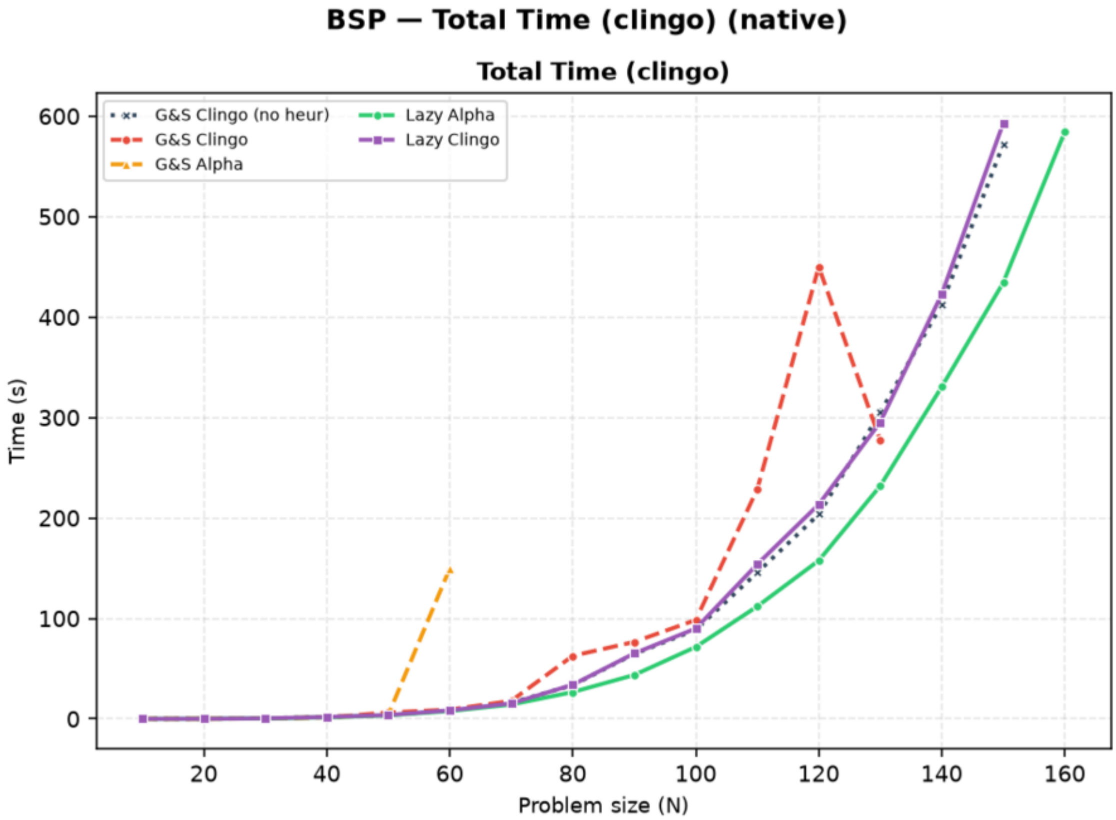

Figure 2: BSP, native backend: total clingo time. Curves are medians over solved instances; a curve ends where its variant stops solving within the limits.

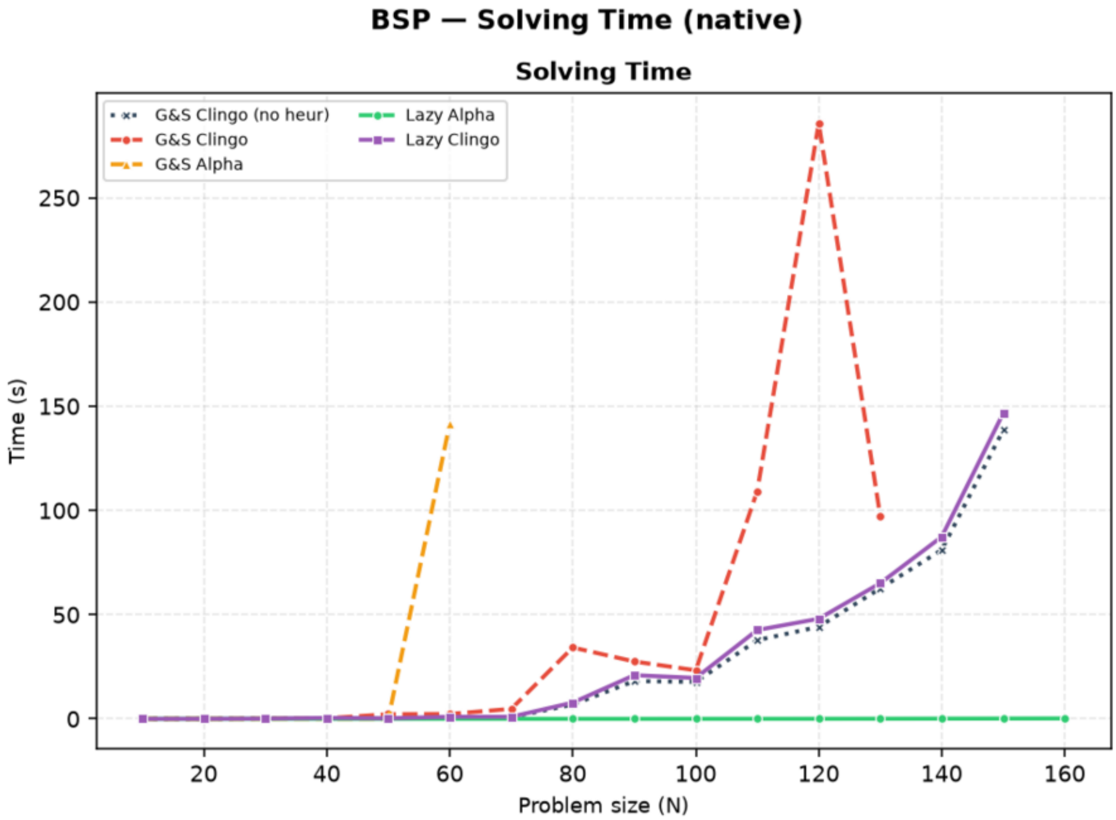

Figure 3: BSP, native backend: solving time only. `la` is indistinguishable from zero at every size; `ga` leaves the chart after $n = 60$.

Variant	Total (s)	Grounding est. (s)	Solving (s)	Choices	Conflicts
`gc_noheur`	203.9	159.4	44.4	78,565	49,993
`gc`	450.3	164.3	286.0	231,260	160,115
`ga`	T	(165.1)	(>434.9)	771,059	563,376
`ga_weak`	230.7	163.1	67.7	121,159	80,847
`la`	157.9	157.5	0.3	120	0
`lc`	213.7	165.4	48.3	78,565	49,993
`la_co`	0.05	0.001	0.00	117	0

Table 6: BSP at $n = 120$, native backend, under the 600 s / 30 GiB limits. T marks a timeout; the parenthesized solving value for `ga` is a lower bound (total minus grounding at the moment it was killed), and its choices/conflicts are the counts reached before the kill. `la_aux` is discussed in Section 4.9.

Three observations follow from Table 6. First, `la` turns search into a formality, 120 choices and zero conflicts, so its total time is the grounding time of the base program: it is the fastest same-encoding variant at every size from $n = 60$ upwards. Under the raised time budget `gc` now finishes at $n = 120$ as well, but at more than twice the cost of `la` (450.3 against 157.9 seconds), and it is `gc`, not `la`, that first crosses into timeout territory as n grows further (Table 2: `gc` fails from $n = 140$, `la` only from $n = 170$).

Second, `ga` is the surprise of the full campaign, and a negative one. Its static unrolling was designed as the faithful ground simulation of the Alpha reading, and Table 5 shows it pays the same $\Theta(n^3)$ grounding as `gc`; but unlike `gc`, whose negative guards deactivate directives as atoms become established, the unrolled directives of `ga` lack those guards by design, so the Domain heuristic is flooded with simultaneously applicable modifications. The search degenerates: at $n = 60$, `ga` needs 149.7 seconds against 8.8 of the baseline, with 1,668,059 choices against 11,810, and from $n = 70$ it never finishes within the limit even at 600 seconds (at $n = 120$ it was killed at 771,059 choices and counting). The faithful simulation is thus *doubly* punished, once in the grounder and once in the solver, so severely that it is the only BSP heuristic variant, ground or lazy, that fails before the heuristic-free baseline does: an instructive datum for anyone hoping to emulate dynamic heuristics in a ground-and-solve pipeline.

Third, Table 6 contains two rows that the figures of this section deliberately omit. `ga_weak`, the cheap approximation of the Alpha reading, stays close to the baseline and is compared against the faithful `ga` in Section 4.8; `la_co`, which changes the problem model itself, is the subject of Section 4.10.

4.5 BSP: the Two Semantics Under the Microscope

Search statistics separate the two semantics sharply, and the full campaign adds an instrumentation-level proof of *why*. The `la` variant makes exactly n choices with zero conflicts at every size: the Alpha reading makes the greedy balance heuristic total, one direct decision per element, which is the known optimal strategy for this problem. The `lc` variant is not merely "close to the baseline": it is *search-identical* to it. At every size, its choice and conflict counts coincide with `gc_noheur` to the last unit (78,565 and 49,993 at $n = 120$).

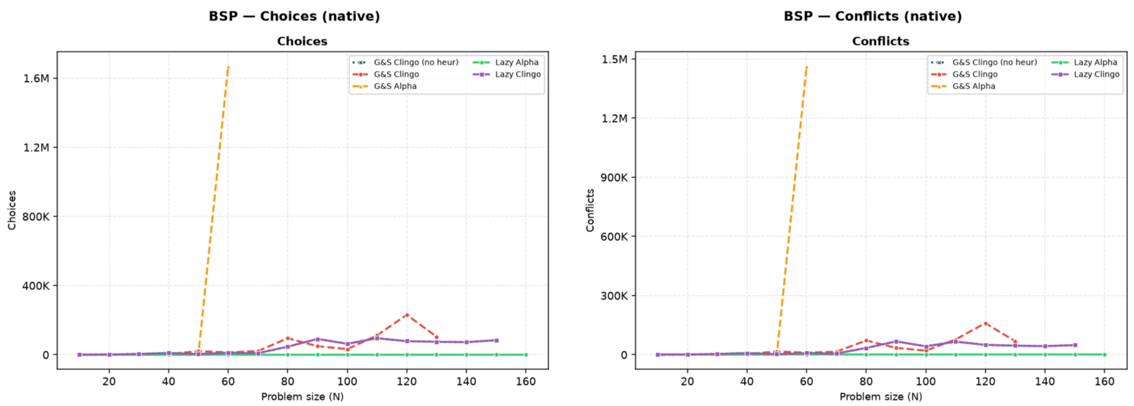

Figure 4: BSP, native backend: choices and conflicts. `la` lies on the horizontal axis; `lc` coincides with the baseline; `ga` leaves the chart after $n = 60$.

The Prolog instrumentation explains the coincidence. On BSP the `lc` rule guards the aggregate source with the clingo reading: the sum over `c/1` becomes usable only when every `c` atom is determined, and `clingo_not(c(X))` additionally requires established falsity. During search these conditions are essentially never satisfied at decision time: the backend reports an average of *zero* applicable candidates per decision (`avg_candidates_per_decide` = 0.0 over 78,565 decide calls at $n = 120$). Every decide call returns nothing, the solver falls back to its default heuristic, and the trajectory reproduces the baseline exactly. The clingo-like lazy variant on BSP is therefore *inert by semantics*: it costs the full decide-loop machinery and buys nothing, which is precisely what the design predicts for a heuristic whose guards wait for information that only exists at the bottom of the search tree.

4.6 BSP: Rules, Variables, and Memory

The compression of the heuristic component is visible in the size of the propositional instance. At $n = 100$, `gc` is fifty times larger in solver variables than every other variant, because its heuristic directives materialize the negative body and the aggregate bound for each (X, S) pair. Notably, `ga` does *not* inflate the variable count (20,402, in line with the baseline): its unrolled directives reuse the same aggregate structures, and its cost shows up as grounding time and unguarded activations instead. The two ground variants thus pay the same combinatorial product in two different currencies.

Variant	**Variables at $n = 100$**
`gc`	1,030,606
`ga`	20,402
`ga_weak`	20,406
`lc`	20,300
`la`	20,300
`gc_noheur`	20,300

Table 7: Number of solver variables at $n = 100$, native backend.

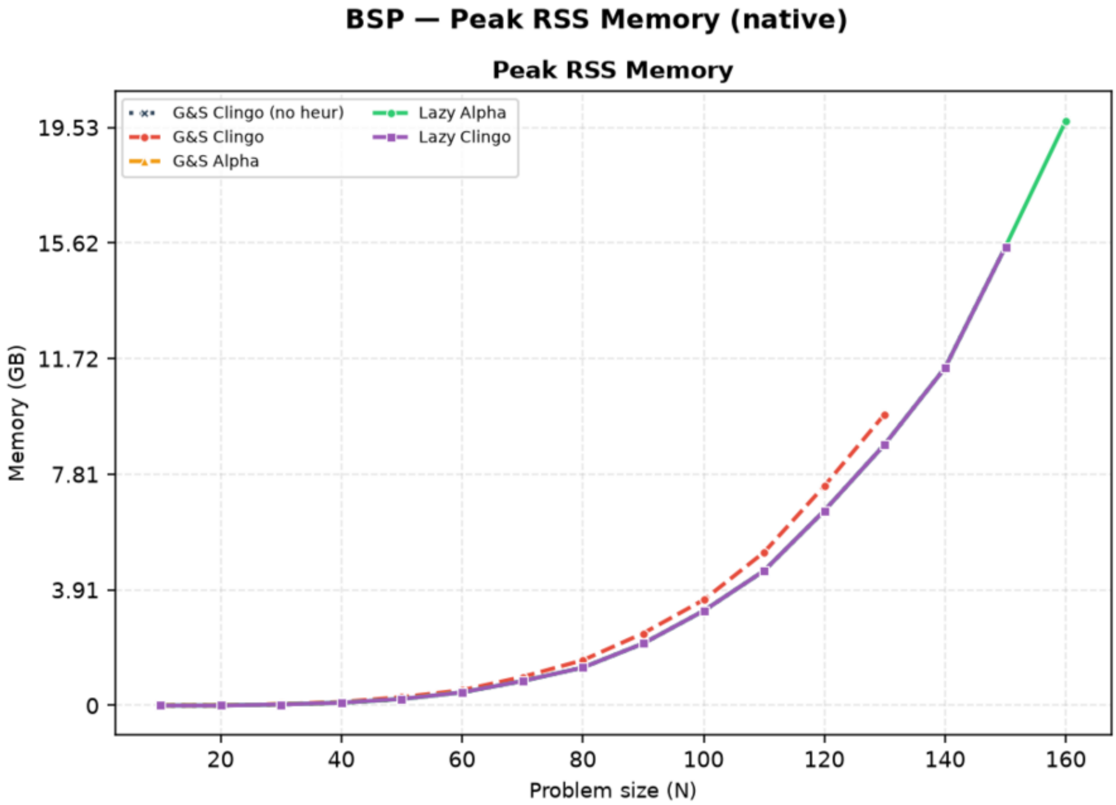

Figure 5: BSP, native backend: peak resident set size.

The memory metric is the peak resident set size of the entire solver process, end to end: it covers grounding, CDCL search, and the custom propagator in a single value, and for the lazy Prolog variants it includes the in-process SWI-Prolog engine and its asserted database. At large n memory is dominated by the base program for every same-encoding variant: at $n = 120$, `gc_noheur`, `la`, and `lc` sit at 6.6–6.7 GB, while `gc` reaches 7.4 GB. Under the 30 GiB budget none of these is close to the ceiling at this size; the base program only approaches it much further out, at $n = 180$, which is where Table 2 records the campaign's only memouts. The heuristic saving is real but bounded by the base program's own footprint; the variant that changes the footprint by orders of magnitude is `la_co` (Section 4.10), because it changes the base program itself. On the Prolog side the SWI engine adds a fixed overhead of the order of ten megabytes, invisible at this scale and dominant only on the trivial `la_co` runs. The RSS *confirms* the saving on the real consumption; the ground-object count of Table 5 *explains its cause*.

4.7 The Price of Laziness: Per-Decision Cost

Intellectual honesty requires stating that the lazy approach does not *eliminate* the cost of the combinatorial explosion: it *moves* it. What ground-and-solve pays once, in memory, at grounding time, the lazy backend pays repeatedly, in time, at every decision. The full campaign turns this from a declared principle into measured numbers, through the instrumentation of the Prolog backend (the native backend implements the same decision loop without per-phase timers, so the Prolog figures are an upper bound for it).

Family	**Variant**	**decide calls**	**cand./decide**	**ms/decide**	**sync (s)**	**overhead**
BSP n=120	`la`	120	121	0.49	0.00	+2%
BSP n=120	`lc`	78,565	0.0	0.18	3.31	+3%
PUP N=160	`la`	197	5.3	3.99	8.82	+18%
HRP N=20	`la`	106	1,248	8.42	0.03	×4.2
HRP N=14	`lc`	73,143	239	3.24	11.05	×145

Table 8: Prolog backend instrumentation on representative solved runs. *cand./decide* is the average number of candidates produced by the `heuristic/4` query per decision; *ms/decide* is the average query time; *sync* is the cumulative cost of mirroring the solver trail into the Prolog database; the last column compares the total time against the same variant on the native backend.

Table 8 decomposes the overhead along its two multiplicative factors: how often the heuristic is consulted (decide calls, driven by the search trajectory) and how much each consultation costs (driven by the number of candidates the rules generate and by the size of the synchronized state). The four corners of the space are all realized in the data. `la` on BSP is the benign corner: 120 consultations of half a millisecond each, a total overhead of about 2% on a 158-second run, so the Prolog and native totals nearly coincide. `lc` on BSP multiplies a *small* per-call cost by 78,565 inert consultations; the query cost stays low enough (under a fifth of a millisecond each) that the resulting overhead is a modest +3% even though the heuristic never contributes a decision, showing that inertness alone is cheap when the per-call cost is small. `la` on HRP multiplies *few* consultations by a large per-call cost, since the HRP reference rules [6] enumerate more than a thousand candidates per decision; on sub-second instances even that turns a 0.42-second native run into a 1.75-second Prolog run (×4.2). `lc` on HRP combines both factors, 73,143 consultations at 3.2 milliseconds each, and the query time alone (over 237 of its 263 seconds) pushes it well past the point where the native backend, running the identical search trajectory without per-phase timers, still finishes in under two seconds; the gap is the largest in the whole campaign (×145). Synchronization, finally, scales with the trail activity and the number of watched atoms: negligible on BSP `la` (a few milliseconds), it becomes the dominant lazy cost on PUP (8.8 seconds at N=160, against 0.8 seconds of query time).

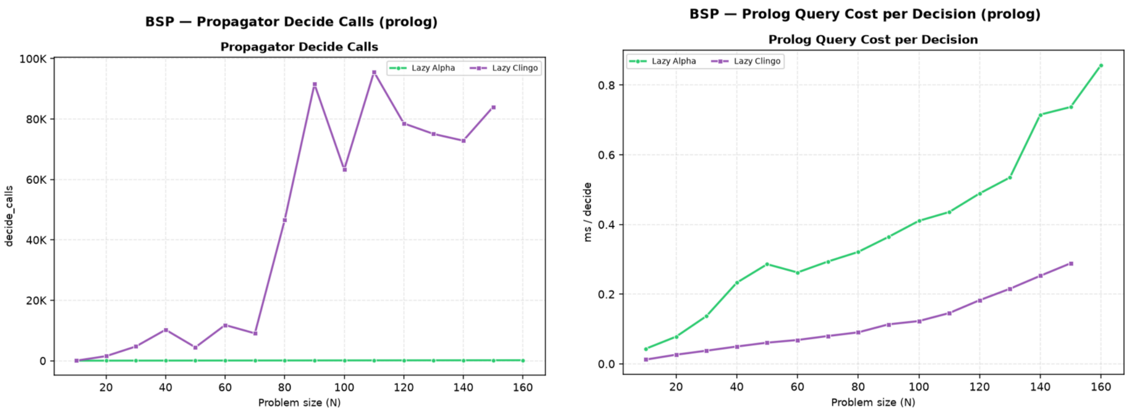

Figure 6: BSP, Prolog backend: decide calls and per-decision query cost. The `lc` curve pairs tens of thousands of calls with a low unit cost; `la` the opposite.

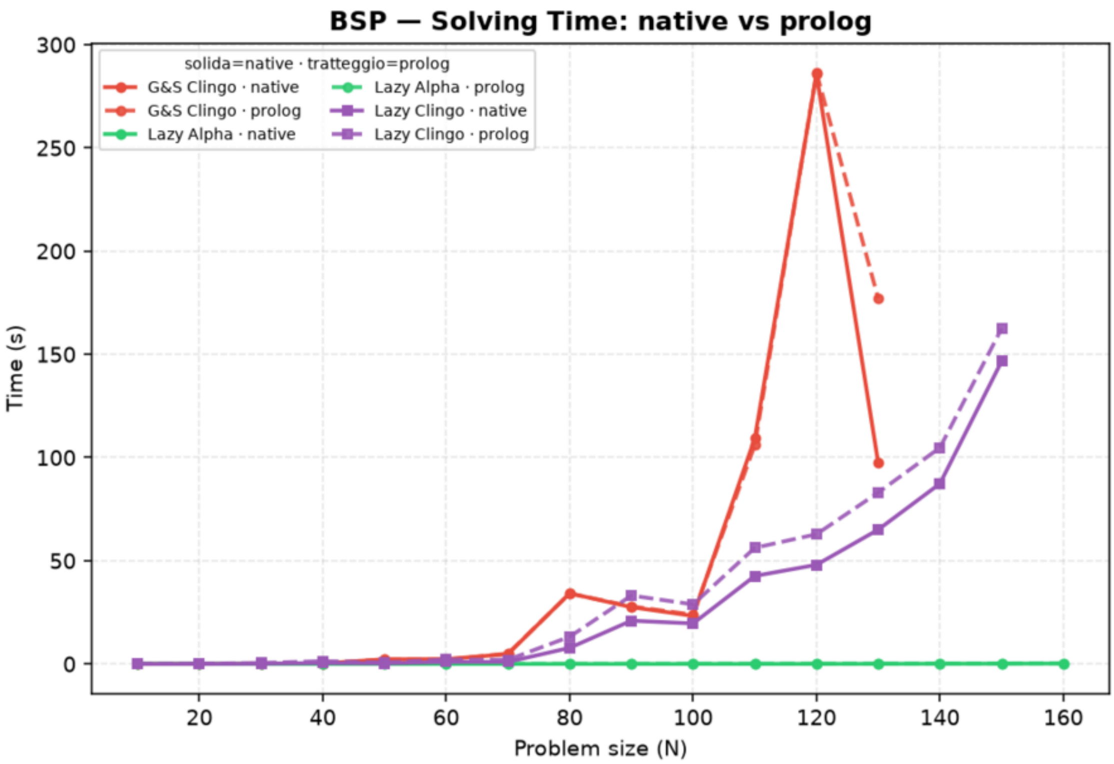

Figure 7: BSP: solving time, native (solid) versus Prolog (dashed) backend, lazy variants and ground reference.

The general law suggested by the table, and consistent with all 520 runs, is that the lazy overhead is the product *decide calls* $\times$ (*query cost* + *amortized sync*), where the first factor is governed by how well the heuristic guides (a good heuristic consults itself n times; a bad or inert one, once per baseline choice) and the second by how much of the program the rules can see. The practical reading is that *lazy heuristics are cheapest exactly when they work*: effective guidance keeps the decision count, and therefore the total overhead, small.

4.8 Two Ground Simulations of Alpha: `ga` versus `ga_weak`

The three subsections that follow analyze the exploratory settings. They are not alternative candidates in the main comparison: each one isolates a single design decision, and its figures include only the variants that decision touches.

The first decision is how faithfully the Alpha reading can be forced into a ground-and-solve pipeline. `ga` is the faithful simulation and pays for both halves of it: it drops the negative literals so that established truth does not deactivate the directives, *and* it statically unrolls the aggregate, replacing the equality binding of S with the monotone bound $S \leq \#\mathtt{sum}\{Y : c(Y)\}$ over every candidate value, so that clasp's max-weight selection reconstructs the current value of the growing sum at solving time (Section 3.1). `ga_weak` keeps only the first half: it treats the negation as Alpha does and leaves the aggregate untouched, which means the grounder evaluates the sum once, on the empty assignment, freezing $S = 0$ and degrading the heuristic to a static ranking of the elements by X.

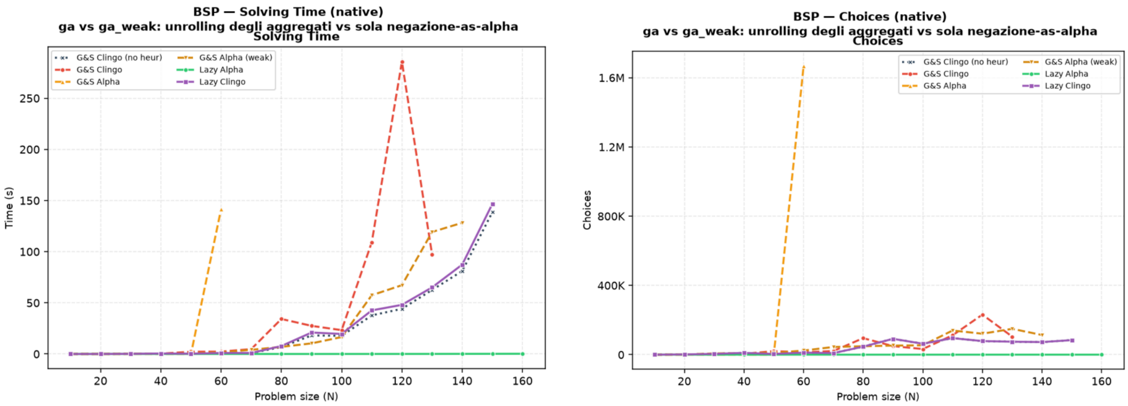

Figure 8: BSP, native backend, exploratory comparison: solving time (left) and solver choices (right) with both Alpha simulations alongside the main variants. `ga` degenerates at $n = 60$ (1.7 million choices, 142 seconds of solving) and times out from $n = 70$; `ga_weak` tracks the baseline over the whole range.

Figure 8 shows what each half costs. The unrolled `ga` explodes exactly where the unguarded activations flood the Domain heuristic: at $n = 60$ it needs 1,668,059 choices and 141.7 seconds of solving against 0.8 seconds of solving (8.8 in total) of the baseline, and from $n = 70$ it never finishes even at the raised 600-second limit, the double punishment already diagnosed in Section 4.4. `ga_weak` sits at the opposite end: structurally it is indistinguishable from the baseline (same rules, constraints, and variables, 20,406 versus 20,300 at $n = 100$, because nothing is unrolled), and behaviourally it stays close to it over most of the range (230.7 versus 203.9 seconds at $n = 120$, with somewhat noisier search: 121,159 choices against 78,565). It solves fourteen of the twenty instances, one size short of the baseline itself (timeout from $n = 150$ against $n = 160$), precisely because it renounces the dynamic part of the heuristic; the frozen $S = 0$ ranking neither guides nor obstructs, so it tracks the baseline's own cost closely without ever improving on it.

Read together, the two simulations bracket the lazy propagator from both sides and confirm that the benefit measured for `la` comes from the *dynamic* aggregate, not from the mere presence of weights or from the Alpha activation rule alone. The half of the Alpha reading that is cheap to ground (`ga_weak`) is behaviourally inert, and the half that reproduces the guidance (`ga`) is combinatorially unaffordable; only the runtime evaluation of the aggregate obtains the guidance without the unrolling, which is the design point the lazy propagator occupies.

4.9 Direct and Auxiliary Forms

The `la_aux` variant routes the lazy heuristic through auxiliary predicates that materialize the heuristic body, including the aggregate value, before the propagator is used. This experiment separates two effects that are otherwise easy to confuse: reducing the number of ground heuristic directives, and avoiding the materialization of the heuristic body itself. `la_aux` keeps only two `__heuristic` facts, but its auxiliaries `h_b(X,S)` and `h_c(X,S)` reintroduce the (X, S) product in the base program, and the full campaign shows the consequences from three angles at once. In the grounder, the auxiliary atoms inflate the instance (145,600 solver atoms at $n = 50$ against 10,352 for `la`). In the propagator, every (X, S) atom becomes a runtime candidate, so the per-decision scan grows with n^2: the native backend already needs 290.3 seconds to solve $n = 60$ (against a fraction of a second for `la`), and from $n = 70$ it times out at the raised 600-second limit; its peak RSS keeps climbing with n regardless (8.7 GB already at $n = 120$, on its way to a memout at $n \geq 160$ where the auxiliary atom count alone exceeds three million). On the Prolog side the same candidate flood must additionally cross the synchronization bridge: at $n = 20$ it already needs 2.9 seconds of cumulative synchronization for 573 decide calls, against 0.3 milliseconds for the 20 calls of `la`; the variant still solves $n = 30$ (367.4 seconds) but times out from $n = 40$. The lesson is clear: if the heuristic body is first materialized into a large auxiliary relation, the intended benefit of laziness is lost on every axis at once, and it is the search cost of the reintroduced product, not memory alone, that dominates the failure. The direct lazy form is the meaningful one.

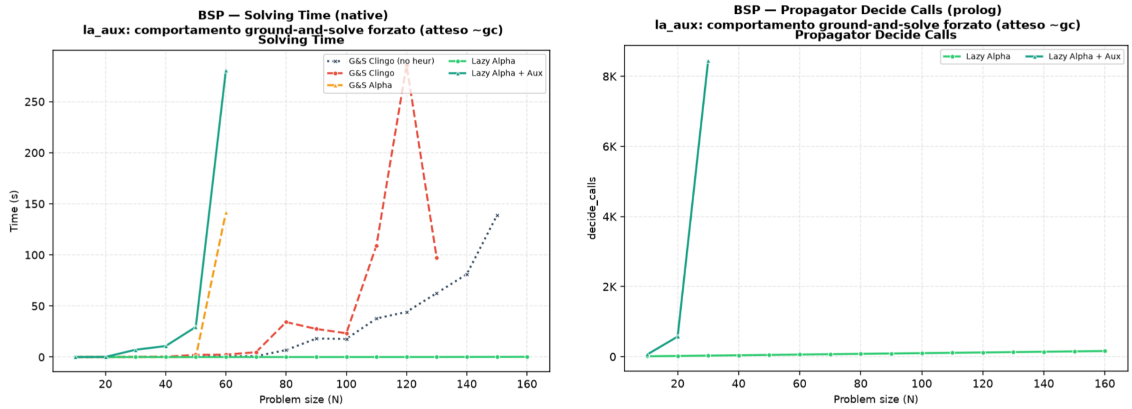

Figure 9: BSP, exploratory comparison of the auxiliary form. Left: native solving time; `la_aux` diverges from $n = 40$ and leaves the chart after $n = 60$, while the direct `la` stays on the axis. Right: propagator decide calls on the Prolog backend; the auxiliary candidates multiply the consultations (573 at $n = 20$, against the n calls of the direct form), and the curve ends at $n = 20$, the last size the variant solves on that backend.

4.10 Effect of the Constraint Formulation

The `la_co` variant changes not only the heuristic representation but also the BSP constraint itself, replacing the quadratic difference check over `sumB`/`sumC` with a single interval constraint over one sum, whose bounds are compile-time constants. The effect is dramatic and worth stating exactly: at $n = 120$ the propositional instance drops from 52,759,258 rules to 366, and the total time from 203.9 seconds (baseline) or 157.9 seconds (`la`) to 0.05 seconds, with the same 117 guided choices and zero conflicts, on both backends (the Prolog run measures 0.13 seconds, the difference being the fixed engine startup). This measurement completes the diagnosis of Section 4.4: once the lazy heuristic has removed the search cost, the *entire* residual cost of BSP is the ground materialization of the balance check, and a formulation that avoids it removes the last Θ-large term. A comparison between `la_co` and `la` therefore measures the interaction between lazy heuristics and problem modelling, not the isolated effect of the heuristic representation, and it is reported as a modelling experiment: its role is to show what the combination "dynamic heuristic + propagation-friendly model" can reach when neither component pays a combinatorial ground price.

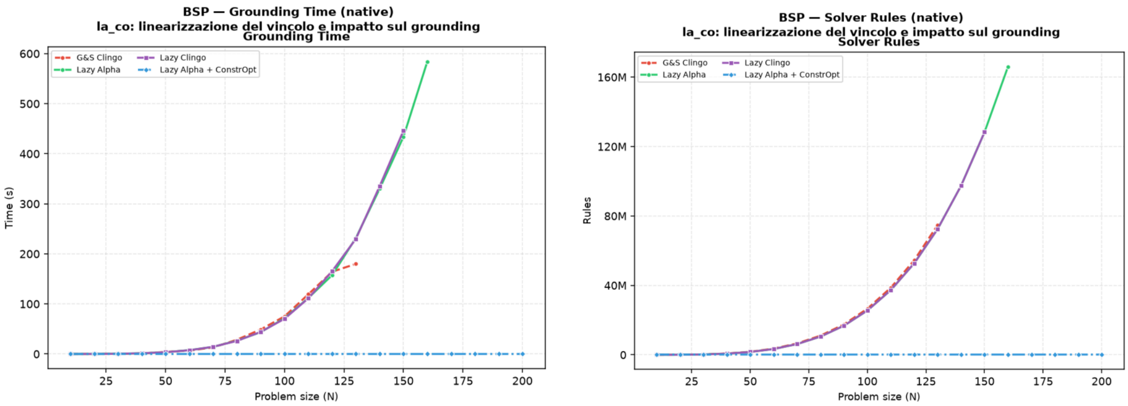

Figure 10: BSP, native backend, exploratory comparison of the constraint reformulation: grounding time (left) and propositional rules (right). `gc`, `la`, and `lc` overlap because they share the quadratic base encoding (52.8 million rules and about 160 seconds of grounding at $n = 120$); `la_co` lies on the horizontal axis of both plots (366 rules, milliseconds of grounding), the visual form of the two-orders-of-magnitude gap discussed in the text.

4.11 PUP: the Grounding Bottleneck at Scale

PUP is the family where the grounding bottleneck, not the search, sets the frontier, and where the lazy representation translates directly into a large saving in time and memory at every common size. Figure 11 and Table 9 summarize the campaign under the 600-second, 30 GiB limits; under this raised budget both Alpha-like variants solve the entire tested range up to $N = 200$, so the frontier that used to separate them has become a cost gap instead.

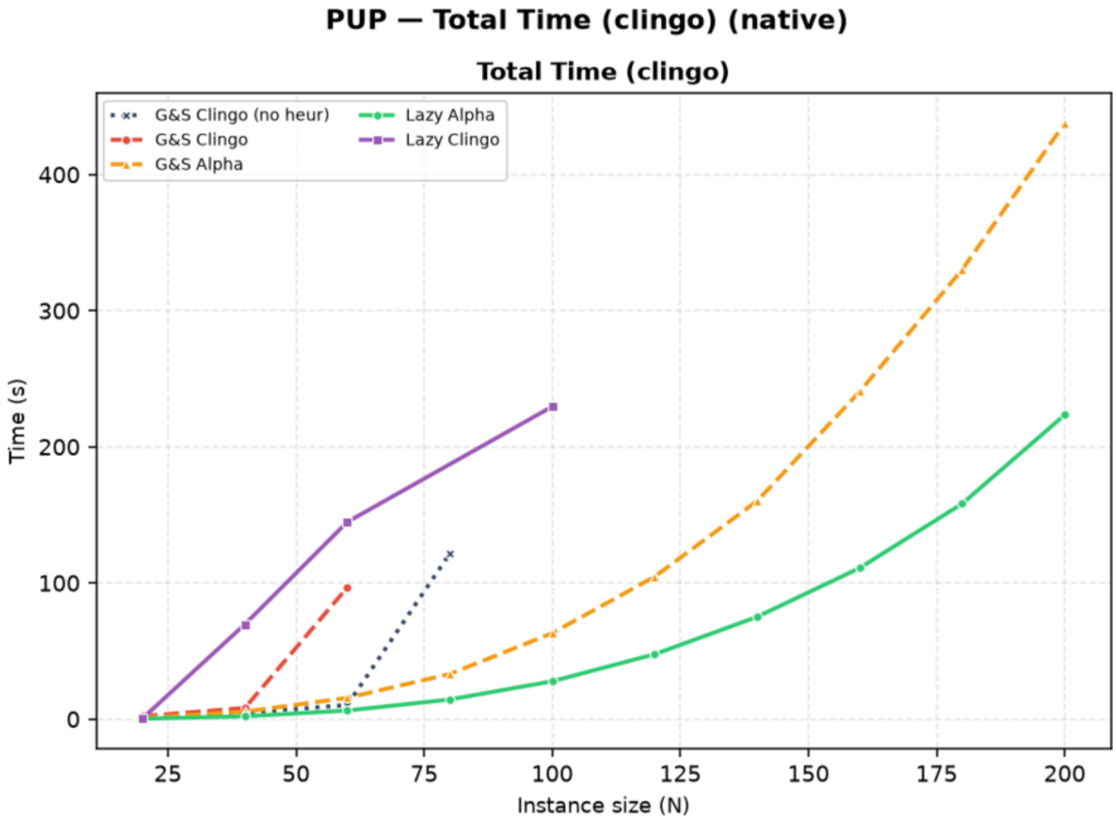

Figure 11: PUP, native backend: total clingo time. Each curve ends at the last instance its variant solves.

Variant	max N solved	Total (s)	Grounding (s)	Peak RSS	first failure
`gc_noheur`	80	121.6	10.2	0.7 GB	T@100
`gc`	60	97.2	18.0	1.3 GB	T@80
`ga`	200	443.8	429.2	25.0 GB	—
`la`	200	226.6	218.0	13.0 GB	—
`lc`	100	230.3	27.0	2.1 GB	T@80 (non-mon.)

Table 9: PUP frontier, native backend, 600 s / 30 GiB per run: largest `double-`N solved and its cost. `ga` and `la` solve the whole tested range and are reported at $N = 200$; the other three still fail inside the range, and are reported at the largest size they solve.

The ordering of the frontiers tells most of the story, and the part that changes with scale tells the rest. The baseline stalls at $N = 80$ by *time*: with no guidance, search explodes first (546,549 choices at $N = 80$, against 272 for `la` at $N = 200$, two and a half times that size). The plain ground heuristic `gc` fares worse still, stalling at $N = 60$: its directives are cheap to ground but do not carry the dynamic aggregate, so they neither prevent the search explosion nor pay for themselves. Both Alpha-like variants clear the whole tested range under the raised budget, but at markedly different cost. `ga` pays for its static unrolling on every instance: at $N = 120$ it needs 106.8 seconds and 7.0 GB against `la`'s 48.6 seconds and 3.4 GB, a 2.2× time and

2× memory gap that widens with N because the unrolled directives grow with the value domains being unrolled, not just with the instance; at $N = 200$ the gap has grown to 443.8 versus 226.6 seconds (1.96×) and 25.0 against 13.0 GB (1.93×). Both variants show the same near-perfect search quality (near-zero conflicts throughout, both semantics-Alpha: 235 choices for `ga` against 162 for `la` at $N = 120$), so the entire gap is attributable to the heuristic's own grounding, exactly as the mechanism predicts. Extrapolating the trend, `ga` is the variant one would expect to hit a memory wall first if the range were pushed further, since its RSS is growing faster than `la`'s and both are approaching the 30 GiB ceiling by $N = 200$; the campaign as run does not observe that wall directly, but the widening ratio is the same signature that, under the previous 8 GiB budget, made `ga` the first of the two to fail.

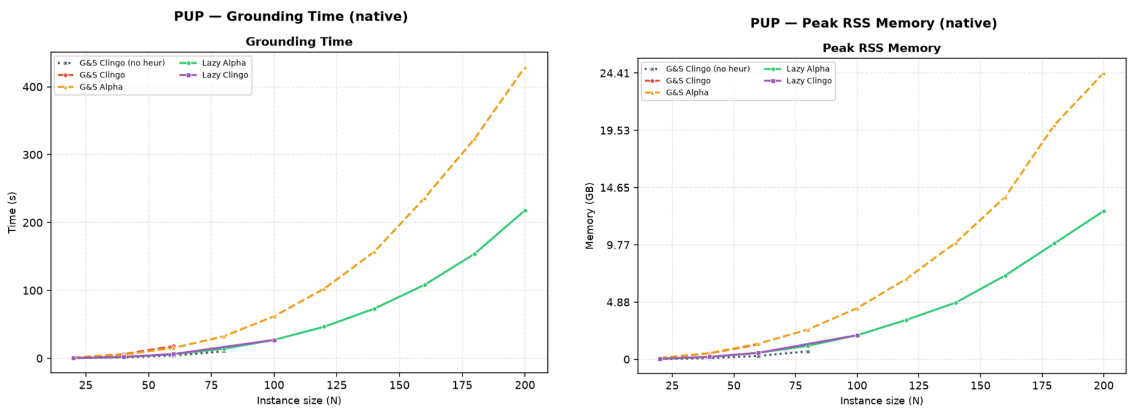

Figure 12: PUP, native backend: grounding time and peak memory. `ga` pays the heuristic unrolling (orange) on top of the base growth; `la` (green) pays the base program only, and stays consistently cheaper at every common size.

The clingo-semantics cells complete the picture from below. Native `lc` oscillates around the baseline, solving $N = 100$ (which the baseline misses) but failing at $N = 80$ (which the baseline solves): as on BSP, its gated aggregates almost never produce candidates (on the instances it solves, the Prolog twin reports *zero* candidates over 190 and 25,197 decide calls). Unlike BSP, however, the native trajectory does diverge from the baseline, so at larger sizes some decisions must eventually fire; those rare, semantically late decisions perturb the default trajectory unpredictably, in both directions. This is the expected behaviour of sporadic, semantically late guidance on a problem with high variance between neighbouring instances, and it is the reason the non-monotone frontier is reported explicitly in Table 3. On the Prolog backend the same inert consultations acquire a price (25,197 calls and 74.5 seconds of synchronization already at $N = 40$), and `lc` collapses to $N = 40$: the pathological corner of Section 4.7 reproduced at scale.

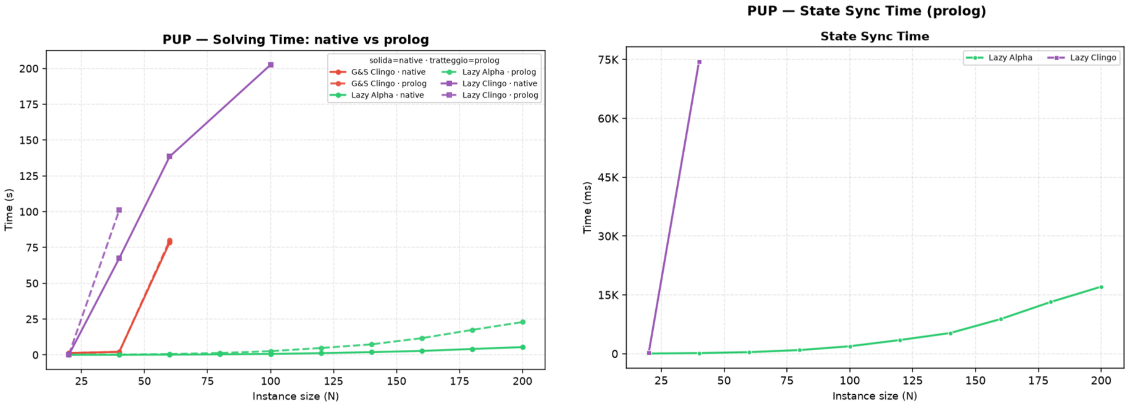

Figure 13: PUP: native-versus-Prolog solving comparison (left) and cumulative synchronization cost of the Prolog backend (right). For `la` the synchronization, not the query, is the dominant lazy cost at scale.

For the well-behaved `la`, the cross-backend comparison isolates the pure cost of the general-purpose engine: identical guidance (both backends solve exactly the same instances with near-identical search statistics), with the Prolog total growing from 112.8 to 133.4 seconds at $N = 160$ (+18%), of which 8.8 seconds are trail synchronization against 0.8 of query time. At PUP scale the bottleneck of the embedded engine is thus *keeping the state*, not answering the queries, which points directly at the batched-synchronization optimization discussed in future work.

4.12 HRP: Semantics Without Aggregates

HRP is the negation-heavy benchmark: its five-level reference heuristic [6] contains no aggregates, so the `la`/`lc` comparison isolates the negation semantics alone. At the benchmark sizes the problem is easy for clasp, every variant except `lc` solves every instance in well under a second on the native backend, which makes HRP useless for frontier claims but ideal as a controlled experiment on guidance quality and per-decision cost.

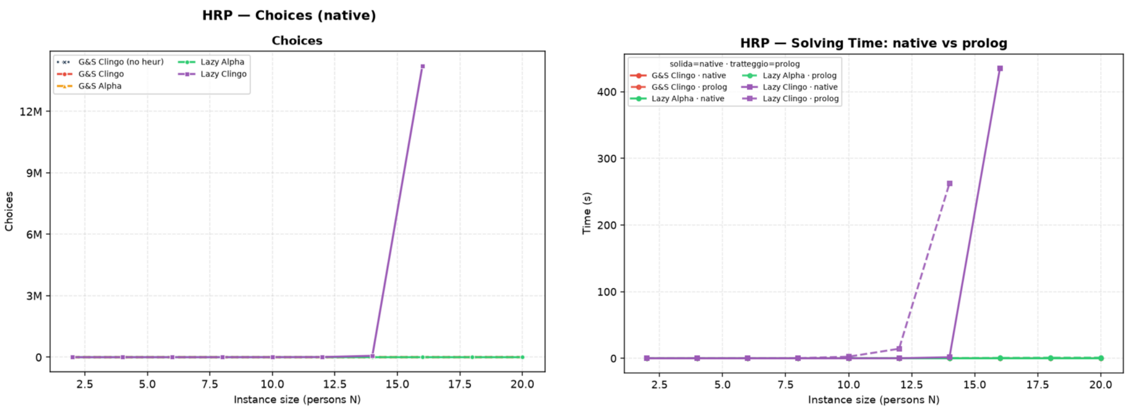

Figure 14: HRP: solver choices on the native backend (left) and the native-versus-Prolog solving comparison (right). The only diverging curve is `lc`, in both plots.

The Alpha reading behaves exactly as designed: `la` tracks `ga` decision for decision (86 versus 88 choices at $N = 14$, zero conflicts for both), confirming on a second problem class that the lazy propagator reproduces the intended Alpha guidance. The clingo reading, by contrast, is *actively toxic* on this heuristic: requiring established falsity on guards such as `not fullCabinet(C)` delays every level of the policy until the corresponding closure information exists, and the surviving low-level modifications steer the solver into a pathological region. Native `lc` needs 66,717 choices and 32,353 conflicts at $N = 14$ (almost eight hundred times the `la` count), still solves $N = 16$ only by spending 435.3 of the 600 available seconds, and times out from $N = 18$; the Prolog twin, one size behind, times out already from $N = 16$ because its per-decision query cost compounds the same exploding choice count faster. The near-coincident frontier on both backends (within one size of each other, instead of exactly equal as under the shorter 300-second budget) still certifies the collapse as a property of the semantics, not of any one evaluation engine: it is the choice and conflict counts, identical on both backends at every solved size, that explode, and the small residual gap in *where* the timeout lands is exactly what a slower per-decision engine predicts on top of an identical, exploding search trajectory. Where BSP showed the clingo reading as inert, HRP shows it as misleading: the two failure modes of "late" information are both on record.

The Prolog backend adds the cost dimension. The reference rules enumerate every applicable (cabinet, thing) and (room, cabinet) pair, more than a thousand candidates per decision at $N = 20$, so even the well-guided `la` pays 8.4 milliseconds per consultation, and the misguided `lc` multiplies 3.2 milliseconds by 73,143 consultations: over 237 of its 263 seconds at $N = 14$ are spent inside `aggregate_all`-free but candidate-rich Prolog queries, anticipating by two sizes the timeout that search alone would cause on that backend. The quantitative gap between the backends on `lc`, 1.81 versus 262.9 seconds at $N = 14$, is the largest in the whole campaign ($\times 145$) and is almost entirely attributable to the per-decision machinery, since the search trajectories coincide at every size both backends solve.

4.13 Native versus Prolog: Summary

Across the full campaign the two backends never disagree on *what* is solved except where the per-decision overhead itself is the binding constraint (`lc` on PUP, $N = 100 \to 40$; `la_aux` on BSP, $n = 60 \to 30$; `lc` on HRP, $N = 16 \to 14$, one size apart instead of exactly coincident now that the raised timeout lets the faster native engine clear one more size before the same exploding search catches up with it). For every ground variant the two systems are byte-identical encodings on the same solver, and their times agree within measurement noise, which doubles as an end-to-end sanity check of the whole pipeline. For the lazy variants the native backend is uniformly at least as fast, with the gap governed by the law of Section 4.7; the Prolog backend remains the more expressive and the more observable one (its instrumentation produced every per-decision number in this chapter), which is the trade-off it was designed to embody.

4.14 Summary of Results

The full campaign supports seven conclusions. First, the lazy representation reduces the ground heuristic component from $\Theta(n^3)$ directives to two constant facts on BSP, and the saving is confirmed on real memory wherever the heuristic grounding is a significant share of the footprint, culminating in PUP where, under a raised 600-second/30 GiB budget that both Alpha-like variants now clear over the entire tested range ($N = 20$ to 200), the lazy variant consistently costs about half the time and half the peak memory of the ground heuristic simulation at every common size, a gap that widens with N. Second, the total runtime does not become constant, because the base program is still fully grounded: on BSP the grounding of the domain encoding dominates every same-encoding variant and is what ultimately stops `la` as well, only two sizes later than under the previous budget; on PUP the base program's own growth is what the lazy variant is left paying once the heuristic's grounding bill is removed. The lazy mechanism isolates the heuristic from the growth of the instance, no more and no less. Third, the semantics axis is not a detail: the Alpha reading turns the BSP heuristic into a perfect greedy strategy (n choices, zero conflicts) and reproduces the intended reference policy on HRP, while the clingo reading is inert on BSP (search-identical to the baseline, with zero applicable candidates per decision) and actively harmful on HRP; representation and semantics must never be conflated. Fourth, the faithful *ground* simulation of the Alpha reading (`ga`) is doubly punished at scale, in the grounder by the unrolling and in the solver by the unguarded activations, on BSP severely enough that it is the only heuristic variant to fail before the heuristic-free baseline does, and on PUP consistently enough that it costs roughly twice the time and memory of the lazy variant at every size both solve; this makes the lazy propagator not merely cheaper but the only practical carrier of that semantics inside clingo at scale. Fifth, laziness is a space–time trade-off whose runtime side is now measured, not merely declared: the overhead factorizes as decide calls times per-consultation cost, it ranges from a few percent (BSP `la`, BSP `lc`) to two orders of magnitude (HRP `lc`), and it is smallest exactly when the heuristic guides well. Sixth, the three exploratory controls delimit the approach from every side: `ga_weak` shows that the Alpha activation rule without the dynamic aggregate is behaviourally inert, `la_aux` shows that materializing the heuristic body forfeits the benefit entirely, and `la_co` shows that pairing the lazy heuristic with a propagation-friendly model eliminates the residual ground cost and solves $n = 120$ in milliseconds. Seventh, the two backends agree on every scientific conclusion while differing exactly where their engineering differs, per-decision cost and observability, so the choice between them is a genuine trade-off rather than a correctness question.

5 Discussion and Conclusions

5.1 Discussion of the Results

The work shows that an important part of the cost of declarative dynamic heuristics can be moved from grounding to search. On BSP, native `#heuristic` directives grow as $\Theta(n^3)$ and exceed one million ground objects at $n = 100$; the lazy variants keep two facts, and the difference is visible in the propositional instance size, in the peak memory, and in the timeout profile. The mechanism generalizes, and the full campaign measures the generalization: on PUP, whose heuristics multiply four unrolled value domains per ground pair, the ground Alpha simulation and the lazy variant both clear the entire tested range under a raised 600-second/30 GiB budget, but the ground simulation consistently costs about twice the time and twice the memory of the lazy variant at every common size, a gap that widens with N because the unrolled directives keep growing with the value domains being unrolled; both dominate the unguided baseline, which still stalls by timeout at less than half their reach. Where the heuristic grounding is the binding constraint, the lazy representation converts directly into a large, growing cost advantage over the best ground heuristic, and into solved instances relative to the unguided baseline; where the base program dominates, as at the top of the BSP range, it converts into an unchanged frontier reached at lower cost, and the residual wall belongs to the domain encoding, not to the heuristic.

This result should not be read as a replacement for general lazy grounding. clingo still grounds the base program; the propagator acts only on the heuristic component. Nor should it be read as a claim that laziness is free: the runtime work is real, it is paid at every decision, and the per-decision metrics exist precisely to keep that cost on the record. The honest formulation is a space–time trade-off with a crossover: below it, ground-and-solve remains competitive, because the explosion is manageable and the lazy machinery (in the Prolog case, an entire in-process logic-programming engine) carries a fixed overhead; beyond it, the constant-size heuristic component is the only representation that survives.

The second central finding is semantic, and it is a finding about *meaning*, not performance. The same declarative heuristic changes behaviour dramatically depending on how partial information is read. Under the Alpha-like reading, the BSP heuristic is a perfect greedy strategy (n choices, zero conflicts) and the HRP policy reproduces its intended level structure decision for decision. Under the clingo-like reading, the very same weights and bodies degenerate in two distinct ways that the campaign now documents separately: on BSP the guards never open at decision time, the propagator reports zero applicable candidates per call, and the search is bit-identical to the baseline (inertness); on HRP the surviving low-level modifications actively steer the solver into a region almost eight hundred times larger, with both backends independently reaching the same pathological order of magnitude of choices even though their exact counts differ (misguidance). Neither reading is "wrong": they answer different questions, and the four-variant matrix exists to keep those questions separate. A comparison that silently mixed the axes, for example by measuring `la` against `gc` and attributing the difference to laziness, would be methodologically invalid; the thesis takes some care never to do so. The campaign adds a corollary about the ground side of the matrix: the faithful ground simulation of the Alpha reading, `ga`, is punished twice at scale, once in the grounder by the unrolling and once in the solver by the activations that its missing guards can no longer switch off, to the point that it is the only BSP heuristic variant that times out before the baseline does. Inside a ground-and-solve pipeline, the lazy propagator is thus not merely the cheaper carrier of the Alpha semantics; at scale it is the only practical one.

The third finding concerns the translation layer between the two heuristic languages. Alpha's weight–priority pairs are global levels; clingo's priorities arbitrate on a single atom. Every encoding that expresses an ordering between different atom families must therefore flatten its levels into weights wherever clingo's reading applies: in the ground variants, because clasp is the consumer, and in the lazy clingo variants, because the propagator deliberately reproduces clingo's behaviour there. This is easy to state and easy to get wrong: an early revision of the lazy-clingo PUP and HRP encodings carried Alpha-style priorities that the clingo semantics does not rank globally, producing an unintended decision order that inverted the designed one (sensors before zones; cabinet-filling before reuse), and diverging from the Prolog twin encodings of the same variants. The final audit caught and corrected the discrepancy by applying the same flattening used in the ground encodings. The episode is instructive: the interaction between priority semantics and evaluation backend is exactly the kind of silent, non-crashing behavioural divergence that a benchmark pipeline cannot detect from exit codes, and it motivates the equivalence and wiring guards that the suite now includes.

A related engineering lesson comes from the silent-fallback failure mode of the Prolog backend. A single missing environment variable degrades every lazy Prolog run to the no-heuristic baseline without any error, since the fallback evaluator simply fails on the Prolog-style rule bodies. The pipeline defends itself with positive evidence of activity (`decide_calls > 0`), cross-backend agreement checks, and a wrapper that forces the variable; the general principle, that a heuristic mechanism must prove it ran rather than merely not fail, seems worth stating explicitly.

Finally, the auxiliary-form experiment delimits the approach from within. `la_aux` shows that laziness only pays if the *entire* dynamic part of the heuristic, aggregate included, stays out of the grounder: materializing the heuristic body into an auxiliary relation reintroduces the very product the mechanism was built to avoid.

5.2 Limitations

The current implementation has explicit technical limitations. The native backend indexes targets and bodies through numeric tuples only; non-numeric arguments are not supported. The arithmetic language covers addition, subtraction, and multiplication; the available dynamic aggregates are sum, count, min, and max, over a single source predicate, with joins delegated to precompiled auxiliary predicates in the encoding. The modifiers `init` and `factor` are recognized but deliberately rejected: in native clingo they modify solver-internal scores that the `decide` interface cannot reach, and approximating them as `level` would silently change their meaning. The propagator is registered sequentially and has no synchronization layer for parallel solving. The tie-break between targets of equal rank is deterministic but does not reproduce clingo's internal scores.

On the evaluation side, the campaign now covers all three families on both backends with the corrected encodings, but its scope must be stated precisely. The per-decision instrumentation exists only in the Prolog backend: the native decision loop is not phase-timed, so cross-backend overhead is measured end to end and the native per-phase profile is inferred, not observed. The HRP instances remain easy for clasp at every benchmarked size, so HRP supports conclusions about guidance quality and per-decision cost, not about frontiers. The PUP frontier claims rest on one instance per size (the `double` family): the non-monotone behaviour observed for `lc` is a reminder that neighbouring sizes differ in hardness, and per-size variance is not measured. The memory metric is end-to-end process RSS, which is the honest but blunt instrument discussed in Sections 3 and 4: it includes the Prolog engine for the lazy Prolog variants and cannot separate grounding memory from search memory. The auxiliary clingo evaluator of the Prolog build re-grounds a program per decision and is a functional reference, not a performance-comparable backend. Finally, the whitelist of static predicates and the inference of the program constant n in the Prolog backend are benchmark-specific conveniences that a production version should replace with a declarative interface.

5.3 Implications

The main implication is that declarative heuristics can be treated as an intermediate layer between modelling and solving. One does not have to choose between fully grounded `#heuristic` directives and procedural heuristics hard-coded in the solver: a compact declarative surface, evaluated at runtime with access to the partial assignment, is a workable third point in the design space, and it can host more than one evaluation semantics behind one syntax. The two backends demonstrate two implementation strategies for that layer, a compiled, incremental one and an embedded general-purpose engine, with the expected trade-off between per-decision efficiency and expressive freedom of the rule bodies.

From an engineering perspective, the work also shows that clingo's propagator and heuristic interfaces are flexible enough to prototype non-trivial guidance mechanisms without touching the solver core: watched literals, incremental aggregate states, per-target modifier resolution, and a global decision ranking are all expressible against the public API of `Clingo::Heuristic`.

5.4 Future Work

Several extensions follow naturally. The mechanism should become configurable rather than always-on, and the expression language deserves integer division, modulo, and comparisons, together with a principled treatment of non-numeric symbols through a stable internal mapping. Aggregates with in-template joins would remove the need for precompiled auxiliary predicates. On the Prolog side, the synchronization protocol could move from per-literal goals to batched updates, and the backend could expose its per-decision statistics through clingo's statistics interface instead of stderr parsing. A parallel-solving synchronization layer and a tie-break policy that cooperates with the solver's own scores are natural solver-side improvements.

On the evaluation side, the PUP synchronization numbers make the batched-update optimization concrete and measurable: at `double-160` the trail mirroring costs eleven times the queries it serves, so a protocol that coalesces retract–assert pairs per propagation batch has a quantified target to beat. Beyond that, configuration, scheduling, and packing domains are natural candidates for broadening the evaluation, precisely because their useful heuristics are dynamic; PUP would additionally benefit from multiple instances per size (the `doubleV` family) to put variance bars behind the frontier claims. A further direction is a verified converter from annotated `#heuristic` directives to lazy facts, with the flattening rule of Section 3.1 applied mechanically rather than by hand; the audit episode discussed above is a concrete argument for automating that step.

5.5 Conclusions

This thesis presented a lazy evaluation mechanism for declarative domain-specific heuristics in clingo, instantiated in two backends over a common propagator design: a native C++ backend interpreting compact `__heuristic` facts with incremental dynamic aggregates, and an in-process SWI-Prolog backend evaluating heuristic rules as logic programs against a synchronized image of the solver trail. The mechanism supports two explicit readings of partial information, the clingo-like and the Alpha-like semantics, and reproduces clingo's weight, priority, and modifier behaviour where that reading applies, including the systematic flattening of Alpha's global priority levels into clingo weights.

The full campaign on BSP, PUP, and HRP under uniform limits (600 seconds, 30 GiB) shows a reduction of the ground heuristic component from $\Theta(n^3)$ directives, more than one million at $n = 100$, to two constant facts, with the corresponding saving in real memory wherever the heuristic grounding matters; on PUP the lazy variant reaches more than twice the solvable frontier of the unguided baseline, and, over the range where both variants succeed, consistently costs about half the time and half the memory of the best ground heuristic. The campaign also quantifies the price: a per-decision runtime cost that ground-and-solve does not pay, which factorizes into consultation count times consultation cost, vanishes when the heuristic guides well, and dominates when it does not. The suite validates that all variants and both backends preserve the answer sets and guards against silent misconfiguration.

The final result is a working, documented, and measurable prototype together with a methodology: two orthogonal axes that keep representation and semantics separate, a flattening rule that makes Alpha-style encodings meaningful under clingo's reading of priorities, and an experimental discipline that states what each number measures. It does not solve the grounding bottleneck in general, but it provides a concrete, verifiable path for bringing dynamic declarative heuristics into clingo without always paying the cost of their full ground materialization.

Bibliography

[1] Martin Gebser et al. *Answer Set Solving in Practice*. Morgan and Claypool Publishers, 2012.

[2] Martin Gebser et al. *Potassco Guide*. Second edition, version 2.2.0. University of Potsdam. 2019. URL: `https://potassco.org`.

[3] Matthew W. Moskewicz et al. "Chaff: Engineering an Efficient SAT Solver". In: *Proceedings of the 38th Design Automation Conference*. 2001.

[4] Martin Gebser et al. "Domain-Specific Heuristics in Answer Set Programming". In: *Proceedings of the Twenty-Seventh AAAI Conference on Artificial Intelligence*. 2013, pp. 350–356.

[5] Carmine Dodaro et al. "Combining Answer Set Programming and Domain Heuristics for Solving Hard Industrial Problems". In: *Theory and Practice of Logic Programming* 16.5-6 (2016), pp. 653–669.

[6] Richard Comploi-Taupe et al. "Domain-Specific Heuristics in Answer Set Programming: A Declarative Non-Monotonic Approach". In: *Journal of Artificial Intelligence Research* 76 (2023), pp. 59–114.

[7] Antonius Weinzierl. "Blending Lazy-Grounding and CDNL Search for Answer-Set Solving". In: *Logic Programming and Nonmonotonic Reasoning (LPNMR 2017)*. Vol. 10377. Lecture Notes in Computer Science. Springer, 2017, pp. 191–204.

[8] Richard Comploi-Taupe, Gerhard Friedrich, and Tilman Niestroj. "Dynamic Aggregates in Expressive ASP Heuristics for Configuration Problems". In: *Proceedings of the 25th International Workshop on Configuration (ConfWS 2023)*. Vol. 3509. CEUR Workshop Proceedings. CEUR-WS.org, 2023.

[9] Jan Wielemaker et al. "SWI-Prolog". In: *Theory and Practice of Logic Programming* 12.1-2 (2012), pp. 67–96.

[10] Michael Gelfond and Vladimir Lifschitz. "The Stable Model Semantics for Logic Programming". In: *Proceedings of the Fifth International Conference and Symposium on Logic Programming*. MIT Press, 1988, pp. 1070–1080.

[11] Martin Gebser et al. "Multi-shot ASP solving with clingo". In: *Theory and Practice of Logic Programming* 19.1 (2019), pp. 27–82.

[12] Martin Gebser et al. "clasp: A Conflict-Driven Answer Set Solver". In: *Logic Programming and Nonmonotonic Reasoning (LPNMR 2007)*. Vol. 4483. Lecture Notes in Computer Science. Springer, 2007, pp. 260–265.

[13] Markus Aschinger et al. "Optimization Methods for the Partner Units Problem". In: *Integration of AI and OR Techniques in Constraint Programming (CPAIOR 2011)*. Vol. 6697. Lecture Notes in Computer Science. Springer, 2011, pp. 4–19.

[14] Gerhard Friedrich et al. "(Re)configuration using Answer Set Programming". In: *Proceedings of the IJCAI 2011 Workshop on Configuration*. Vol. 755. CEUR Workshop Proceedings. CEUR-WS.org, 2011, pp. 17–24.

[15] Armin Biere. *runlim: a tool for sampling time and memory usage of a process.* `https://github.com/arminbiere/runlim`. Accessed 2026.

Acknowledgements

I would like to thank Prof. Carmine Dodaro for his guidance at the University of Calabria, and Prof. Torsten Schaub and Dr. Javier Romero for their supervision and support during my Erasmus period at the University of Potsdam. Their feedback helped shape both the modelling choices and the implementation decisions behind this work.

I am also grateful to the ASP and Potassco communities. Tools such as clingo make it possible to experiment with advanced research ideas while relying on a solid, well documented, and open foundation.

Finally, I thank my family and the people who stayed close to me throughout this thesis work, for their constant support and patience during the many long debugging sessions.